

Aangan

Kaavish Report
presented to the academic faculty
by

Faraz Ahmed Khan	fk03983
Hareem Feroz Ahmed	hf04097
Huda Feroz Ahmed	ha04081
Kainat Abbasi	ka04051
Mudasir Hanif Shaikh	ms03831



In partial fulfillment of the requirements for
Bachelor of Science
Computer Science

Dhanani School of Science and Engineering

Habib University
Spring 2020

Aangan

This Kaavish project was supervised by:

Dr. Syeda Saleha Raza
Faculty of Computer Science
Habib University

Approved by the Faculty of Computer Science on _____.

Dedication

This project is wholeheartedly dedicated to our beloved parents who have not only raised and nurtured us but have also taxed themselves dearly over the years of our education and intellectual development. A special feeling of gratitude to our parents for their endless love, sacrifices, prayers, support, and advice. To our siblings and friends, who shared their words of advice and encouragement to finish this project.

Most importantly, the project is dedicated to the spirit and strength of our group Aangan, where everyone sincerely uplifted and encouraged each other in times of despair and spent countless hours reflecting to keep the group's spark strong.

And lastly, we dedicate this project to the Almighty God, thank you for the guidance, strength, power of the mind, protection, and skills, and for giving us a healthy life in these testing times of the pandemic. All of these, we offer to you.

Acknowledgements

Team Aangan would like to extend its gratitude to our supervisor and mentor, Dr. Saleha Raza for her constant support throughout the project in these difficult times amidst the pandemic. Her experience and support enabled us to achieve our deliverables and gain a deeper understanding of the problem.

We would like to thank our friends, family and acquaintances who helped the project in their capacity by participating in the process of data collection and scoring by providing samples for each character and manually grading anonymous samples. From getting their family members on-board to handing over their mobile phones to kids in the family and neighborhood, they have played a significant role in introducing diversity in our data. We are truly thankful for their keen interest and enthusiasm in the project and the underlying idea behind it.

We are extremely thankful to Mrs Nazish Hasan for helping us bridge the tiny gaps in the learning process being enabled by the application in its initial stages. Her insights and feedback on the education landscape for children helped us in bringing a lot of improvement in the project and in making it more relevant to the target audience.

We thank the CS program, the Kavish Working Group and Habib University for providing us the technological support and resources required for the research. We would also like to thank Mudasir Hanif Shaikh from the Class of 2021 for his role as a student representative in enabling smooth communication between the students and the Kavish Working Group throughout the project.

Abstract

The use of artificial intelligence and machine learning for educational purposes has increased significantly over the past few years. This project proposes a novel e-learning mobile application for Urdu writing, a language spoken by 200 million people around the world, and for drawing. The proposed application allows learners to complete writing exercises and provides feedback on the attempted exercises. The application uses machine learning and statistical models to generate the feedback score. The project also includes the collection of a novel dataset for Urdu characters. The application and explored research hold tremendous potential sets a pathway for further research in automatic scoring and stroke-based analysis of Urdu. Moreover, the application lays a pathway to introduce more e-learning applications in Urdu and drawing.

Contents

1	Introduction	12
1.1	Problem Statement	12
1.2	Proposed Solution	13
1.3	Intended User	14
1.4	Project gantt chart and deliverable	14
1.4.1	Gantt Chart	14
1.4.2	Project Deliverable	15
1.5	Key Challenges	16
2	Literature Review	18
2.1	Hand-writing Analysis of Urdu	18
2.1.1	Literature Researched	21
2.2	Analysis of Hand-drawn Sketches	22
2.2.1	Literature Researched	24
2.3	Related Existing Solutions	25
3	Software Requirement Specification (SRS)	29
3.1	System Overview	29
3.1.1	Module: Teaching Urdu	29
3.1.2	Module: Drawing	31
3.1.3	Performance Reporting and Visualization	31
3.2	Functional Requirements	31
3.3	Non-functional Requirements	33
3.3.1	Performance	33
3.3.2	Reliability	34
3.3.3	Availability	34
3.3.4	Maintainability	34
3.3.5	Scalability	34

3.4	External Interfaces	34
3.4.1	User Interfaces	34
3.4.2	Application Program Interface (API)	39
3.5	Use Cases	39
3.6	Datasets	42
3.7	System Diagram	44
4	Software Design Specification (SDS)	47
4.1	Software Design	47
4.2	Data Design	49
5	Application	51
5.1	Register, Login and Main Menus	51
5.2	Urdu Module	57
5.2.1	Urdu Letter Writing	57
5.2.2	Urdu Word Writing	59
5.2.3	Object Identification with Urdu Words	60
5.2.4	Urdu Words' Identification	61
5.3	Drawing Module	62
5.4	Personalization of Exercises	63
5.5	Dashboard	63
6	Dataset	65
6.1	Data Collection for Urdu Characters	65
6.1.1	Data Scoring	67
6.1.2	Data Pre-processing	68
6.2	Data Generation for Urdu Words	68
7	Experiments and Results	71
7.1	Methodology	71
7.1.1	Interclass Score	71
7.1.2	Intra-class Score	73
7.1.3	Scoring for Urdu Characters	74
7.1.4	Scoring for Urdu Words	75
7.1.5	Interclass Score	76
7.1.6	IntraClass Score	78
7.1.7	Score Combination	78
7.2	Results	78
7.2.1	Inter-class Model	78

7.2.2	Intra-class Model	79
8	Conclusion and Future Work	82
8.1	Conclusion	82
8.2	Future work	82
	Appendix A Code	84
	References	85

List of Figures

1.1	Gantt chart of the application	15
2.1	The 8-direction stroke convention [22]	19
2.2	Proposed procedure for segmentation of words [22]	20
2.3	Stroke representation of sketches in the QuickDraw dataset	23
2.4	UI of Kaligo Application	27
2.5	UI of Kids Urdu Learning App - Alphabets Learning App	28
2.6	UI of Draw it - Learning App	28
3.1	Introductory Screens	35
3.2	Module Selection	36
3.3	Urdu Module - UI	37
3.4	Remaining modules for Urdu	38
3.5	Drawing module	38
3.6	Performance Visualization Screens	39
3.7	Usecase diagram of the application	41
3.8	Usecase diagram of Urdu module	42
3.9	UHaT: Urdu handwritten text dataset	44
3.10	System Diagram	45
4.1	UML Class Diagram of the system	47
4.2	Entity Relationship Diagram of the system	50
5.1	Logging in to application	52
5.2	Urdu letter tracing	53
5.3	Registering to use application	54
5.4	Adding child profile	55
5.5	Selecting child profile	56
5.6	Selecting module	56

5.7	Selecting Urdu module	57
5.8	Urdu letter tracing	58
5.9	Urdu letter tracing for incorrect attempt	59
5.10	Urdu word tracing	60
5.11	Object identification with Urdu Words	61
5.12	Urdu Word identification with Objects	62
5.13	Drawing	63
6.1	Distribution of Characters	66
6.2	Data Collection Application	66
6.3	Interface for Scoring Application	67
6.4	Distribution of Scores	67
6.5	Distribution of Age Groups	68
6.6	Word Data Samples	69
6.7	Word Data Samples	70
6.8	Data Generation Process	70
7.1	From Left to right: Urdu characters Chey (Primary features: Horizontal Line left to right, Semi Circle Up to down. Secondary Features: 3 dots, middle), Bey (Primary features: Start Vertical Line, Horizontal Line right to left, End Vertical Line. Secondary Features: 1 dots, below), and Ayn (Primary features: Start Semi Circle, Semi Circle Up to down. Secondary Features: None)	72
7.2	Primary Features Model Architecture	72
7.3	Secondary Features Model Architecture	72
7.4	Urdu Words Segmentation Model Architecture	75
7.5	The figure shows the process of segmentation. From left to right: Reference image, sample stroke, sample stroke with dots removed, sample stroke segmented into two different strokes to be sent to the model described in Section 7.1.2	75
7.7	Stepwise procedure for the Dataset Preparation	77
7.8	Few Samples from the training set	77
7.6	Architecture for the Drawing interclass Model	77
7.9	From left to right: Reference Image shown on the application for the character ‘Jeem’, Sample stroke classified as ‘unrecognizable’ by the interclass model, Sample stroke classes as ‘recognizable’ by the interclass model.	79

7.10	From left to right: Reference Image shown on the application for character ‘Laam,’ Sample stroke which was given a score of 53 while annotated score was 60, Sample stroke which was given a score of 97 while annotated score was 90.	80
7.11	From left to right: Reference Image shown on the application for character ‘Laam,’ Sample stroke which our trained model gave a score of 72 while annotated score was 90, Sample stroke which was given a score of 67 while annotated score was 70.	81

List of Tables

7.1	The table shows the accuracy of the proposed model on a few random categories along with overall accuracy averaged over all categories . .	79
7.2	M.A.D: Mean Absolute Difference, RA: Range Accuracy - Summary of the results of the statistical model proposed in 6.2.1.	80
7.3	M.A.D: Mean Absolute Difference, RA: Range Accuracy - summarizes the results of the regression models proposed in 7.1.2	81

1. Introduction

1.1 Problem Statement

Artificial Intelligence, machine learning, and the development of new tools and applications have impacted all spheres of life. Education and e-learning have also significantly benefited from these developments, but as for all technologies, not all regions of the world and not all domains have been able to benefit from these technologies to their true potential. Considering Pakistan, it is not only far behind in advancements in e-learning technologies, but according to a survey on early childhood education in Pakistan, [6] Pakistan is also far behind in meeting the United Nation’s sustainable development goals of education. Some primary reasons for it are lack of learning resources, lack of adequate feedback provided to learners, and lack of advanced teaching methodologies. Moreover, even though Urdu is the national language of Pakistan and is spoken by 200 million people around the world, there exist very limited educational resources to learn it and it is still under-researched in the domains of machine learning and artificial intelligence.

Based on the understanding of the educational landscape of Pakistan, we identified that there is a need to focus on the education of young learners and develop an intuitive online learning application for the education of Pakistani children including non-native Pakistani children living abroad. The application developed is the first of its kind and is focused on teaching Urdu language and drawing to children. This project also explores the potential in stroke and handwriting analysis to teach Urdu handwriting. First, it proposes a novel dataset of Urdu characters and words, machine learning, and statistical estimation models, which can assess and provide feedback on writing exercises. It introduces an application that is the first of its kind that incorporates these models to the learner. The application also creates a personalized experience by identifying the learner’s weaknesses and focusing on exercises on which the learner has made repeated mistakes. Most importantly, following

the Covid-19 pandemic, the application can prove beneficial to enhance the learning experience for children by providing them timely feedback and personalized exercises for practice. The research presented in this paper can be extended to other languages with a script similar to Urdu, such as Arabic, Persian and Punjabi languages.

1.2 Proposed Solution

The project aims to develop a learning tool in form of a mobile application that can not only be a great learning resource for children, but can also provides constructive feedback in order to enhance individual performance. The aim of the project is to provide a learning platform for young children that assists parents (or guardians) in tutoring their children at home. The application provides personalized learning experience to learners and makes them practice more with aspects they're generally known to struggle with or are doing wrong unknowingly.

The main modules of the application include Urdu and Drawing. The **Urdu module** assists in writing Urdu letters and Urdu words from scratch by following an approach similar to Urdu Qaidah. This module also helps create a thorough recognition of Urdu letters and words using flashcards.

The idea behind both the **Drawing module** is to encourage creativity and make it easier for learners to practice art outside their school. The drawing module helps to draw from scratch and then gradually increases the difficulty level extending to more complex figures.

The most integral part of the project is **scoring and feedback**. At every exercise attempt made by the learner, the app would analyze the attempt and generate a score. This score will then be used to provide feedback to the learner. The analysis mechanism would save the record of exercises that the learner struggles the most with and use it to provide him with similar exercises to practice with in successive attempts. The application also reports the performance of a learner by providing a **visualization of individual progress** at every stage. The performance can be accessed from the dashboard for every individual user.

1.3 Intended User

As outlined in the problem statement above, we are primarily targeting kids in the age bracket of 3 to 7 years. Children would require the supervision of parents and/or their guardians for two reasons: one to assist them in accessing the application and secondly for their parents to keep track of their progress. In that sense, the intended users of our app are children in the aforementioned age bracket and their parents. We may extend our user base to include schools but that is not our immediate goal. Currently, our app aims to serve as a helpful tool for those parents that want their children to learn and appreciate their national language while at the same time allowing for their cognitive development through personalized exercises for Urdu, and drawing.

1.4 Project gantt chart and deliverable

1.4.1 Gantt Chart

Figure 1.1 shows the gantt chart for the application. For better visibility the Gantt chart can be accessed [here](#).

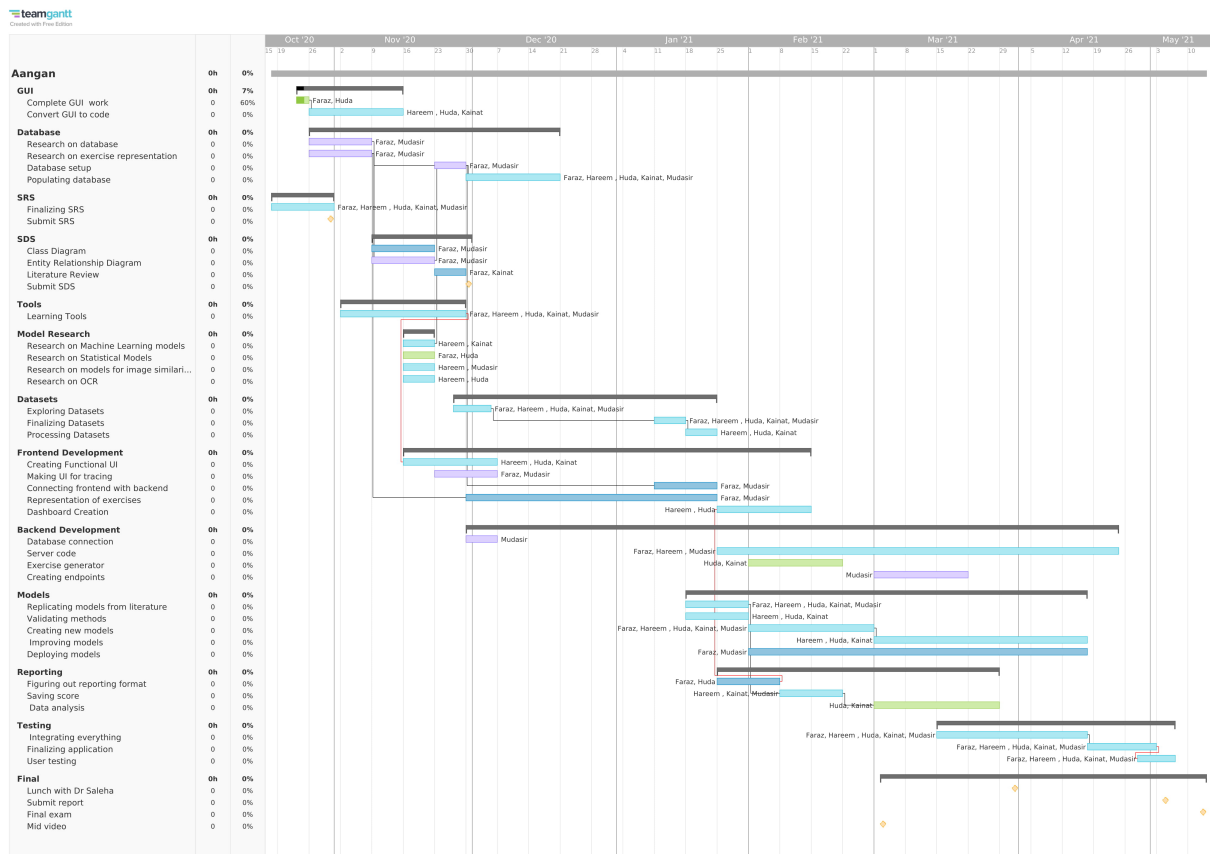


Figure 1.1: Gantt chart of the application

1.4.2 Project Deliverable

What will be covered as a part of the project

1. A Database that stores all information that the application is designed to store. This includes practice exercises, tests, scores and an estimate of time taken.
2. Efficient models for scoring, feature extraction, generating exercises and similarity measurement that make use of Machine Learning and/or Statistics.
3. An Application wire-frame that signifies how the product works as a whole.
4. A SRS document listing the details of the application/project.

5. A SDS document listing design details and how the application is built.
6. An initial proposal that states the aims, objectives and goals of the team.
7. Literature review
8. Product Prototype
9. A Poster summarizing the features of the final product.
10. A Final report with details of the end product

What will *not* be covered as a part of the project

1. The application provides a real-time interactive learning experience by acting as a tutor to a learner. Video lessons are therefore not a part of the application.
2. The application is not intended to test or improve a learner's speaking ability. The methods and means employed in the application only cover a range of cognitive levels including writing, comprehension and understanding.

1.5 Key Challenges

1. Online and offline elements in application

Ideally, most application features and modules should run without being connected to the internet, but keeping everything offline would mean application size can become huge. Similarly, keeping application features and modules available to use only when connected to internet, not only limits accessibility but Amazon Web Services free tier provides limited access only. Therefore, deciding which elements to keep online and offline will be a challenge.

2. User experiences on different devices

We aim that the the application should be able to run smoothly on most touch-screen systems. However, different systems have different touch sensitivities (touch sampling rate). Moreover, different devices have different refresh rates¹ which means that user experience may vary depending upon the device being used.

¹refresh rate: the number of times the monitor updates with new images each second

3. Exercises Representation

We have access to the learning material and exercises through various textbooks employed in schools, however efficient methods of representation in the database of the application is a major challenge.

2. Literature Review

The Urdu and drawing module of our application is to teach Urdu handwriting and basic sketching respectively to children using online handwriting recognition technology. The modules would detect if the child has written or drawn correctly and will assign a score to the child's attempt as feedback. The research with regards to our application spans multiple fronts. This section sheds light on those fronts by focusing on different modules of our application separately. The section then also gives a general overview of existing applications and solutions which exist for our problem.

2.1 Hand-writing Analysis of Urdu

There are various fronts of research centered on hand-writing and stroke analysis which are extremely relevant to our problem. In this section, we will look at existing work for different languages especially languages that use Arabic script, and will discuss work related to the analysis of handwriting, representation of handwriting in online systems, methods to score stroke quality, and classification and recognition of characters.

Hand-written text is primarily represented as images and different preprocessing techniques are applied to them to remove noise. [26] simply binarised and then morphologically cleaned the images to remove the noise that was produced by the binarization process. Another way to represent and store handwriting is as a stroke sequence. By a "stroke", we mean any sequence of points that are captured once the pen touches the paper until that takes off. [22] represents handwritten words as a stroke sequence that is based on the 8-direction stroke convention which is also shown in figure 2.1. An online Arabic tutor system also represents student's handwriting as strokes and in the preprocessing step, it modifies the characters by removing the points to reduce the number of classes and reorder the strokes [1]. Point removal is

noticeable as just like Arabic, Urdu also has groups of characters that are differentiated by the number and position of dots so eliminating these points leads to the same character and they can be assigned to the same class.

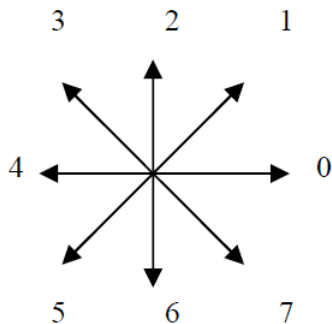


Figure 2.1: The 8-direction stroke convention [22]

The effectiveness of using digital devices to develop handwriting skills was analyzed by [9] and was found to be highly effective. Similar research conducted in Arabic [25] and Chinese [27] was also relatively successful. One of the most sophisticated systems, ‘IntuiScript,’ was proposed by [31], with versions in English and French, which introduces the concept of scoring for cursive writing. Scoring essentially implies assigning a score to the handwritten character/word based on how similar it is to the perfect sample of that character/word. The perfect sample(s) are identified by the domain experts. [31] explicitly mentions that automated scoring of handwritten samples is a different research problem than recognizing handwritten for which significant-quality research exists. However, research for automated scoring is rather scarce and holds tremendous potential, particularly in e-learning applications for handwriting.

For automated scoring, [31] proposes a fuzzy model which combines interclass and intraclass scores to provide a final score between 0 and 1. The proposed models work on features relating to the shape, direction, and order of the sample stroke. For this purpose, they use the HBF49 [11] feature set alongside a fuzzy histogram of orientations. [25] proposes a similar system for Arabic that works on raw features of the stroke, such as stroke order and the number of strokes, and uses a fuzzy model for automated scoring. It uses a highly crude set of features, and hence the possibility of erroneous results is quite high. We were not able to find any relevant research on assessing the quality of stroke in Urdu. For Arabic words, [22] presents a recognition

system based statistical analysis of a typical Arabic text and shows that the sub-word in the Arabic language is the basic pictorial block rather than the word. 2.2 shows a segmentation methodology proposed by [22] to recognize Arabic words.

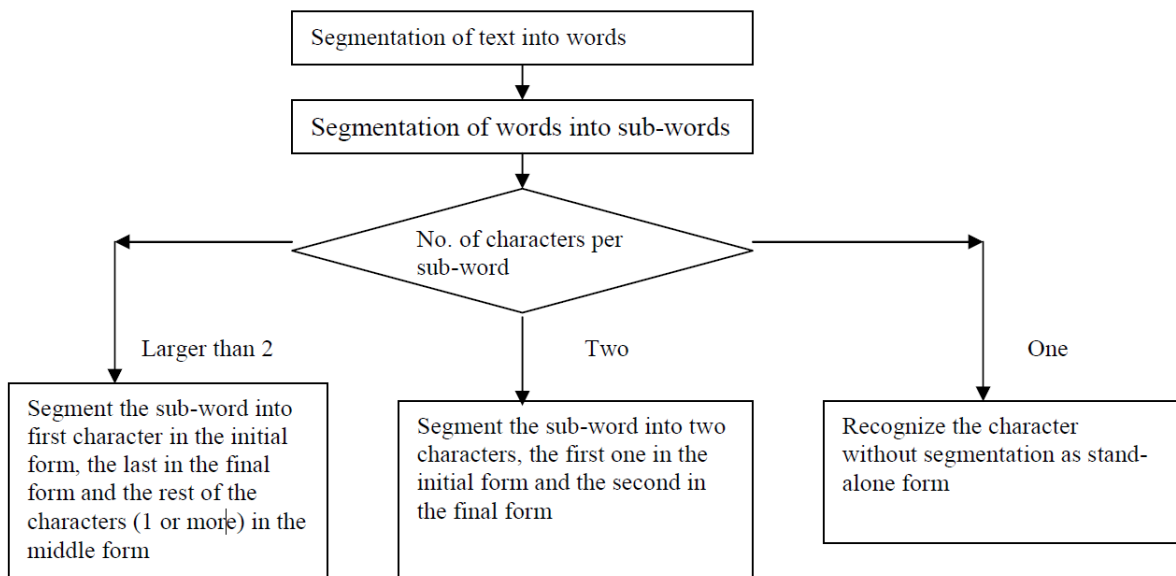


Figure 2.2: Proposed procedure for segmentation of words [22]

While character classification and recognition are a different problem than the problem at hand, as established by [31] as well, the existing literature and existing datasets could still provide helpful insight. [7] proposes a holistic feature set that could be used to recognize Urdu characters and cords by dividing the Urdu characters into groups. Characters within the group only vary by the position and number of dots around the character. The Urdu character set is further divided into two feature sets, primary features, which distinguish between different groups, and secondary features, which distinguish characters within a group. Various machine learning models are trained on these feature sets, which recognize Urdu characters with considerable success. For Arabic, [34] uses Beta-elliptic features [8] on the ADAB dataset [5] to identify different characters using an SVM (Support Vector Machine). Both [7] and [34] work with stroke data (sequence of x,y) for feature extraction and consequently for model building.

There is also significant literature that uses images of stroke for their identification and their recognition. [16] uses uhat dataset [17], which is a collection of scanned

Urdu characters and proposes a CNN which archives an accuracy of 90%. A combination of a deep autoencoder and CNN was proposed by [4] to achieve accuracy in excess of 95%. For Urdu words, a novel dataset and a model were proposed by [3], which works on images of Urdu text captured by digital devices and recognizes the text with considerable accuracy.

Our literature review indicates limited research in Urdu, especially in terms of stroke analysis for automated scoring. The review also suggests a scarcity in research for handwriting analysis in general for the Arabic Script. Moreover, there are almost no publicly available datasets for Urdu characters and words in the form of stroke data.

2.1.1 Literature Researched

The table on the following pages outlines important details acquired from the literature researched.

Research Paper Title	Hand-writing Language	Approach	Dataset	Performance/Accuracy
Arabic Character Recognition using Approximate Stroke Sequence	Arabic	Considers words as a basic pictorial block rather than whole words. Segments text into words, and then words into subwords in three ways: 1) Sub-words in their stand-alone form, 2) Sub-words consisting of two characters, 3) Sub-words consisting of more than two characters.	A sample of Arabic text collected randomly from old books, modern books, newspapers, and other available sources on the web.	Up to 80% accuracy for few single characters in stand-alone form

2.2 Analysis of Hand-drawn Sketches

The drawing module of our application displays a simple sketch to the user and the user is required to draw the sketch on the canvas using a pen. The main task under this module is to be able to recognize how good or bad a user-drawn sketch is in comparison to the reference image. There exists a generous amount of research within machine learning approaches that target hand-drawn sketches. Hand-drawn sketches come with their own share of domain-specific challenges and benefits as compared to natural photos. For instance, freehand sketches are highly diverse, sparse, and abstract and do not follow a specific set of rules. At the same time, hand-drawn sketches do not have any background clutter and the sequential nature of sketch generation allows for better analysis. Sketch properties also allow for various representations of hand-drawn sketches and at the same time call for Machine learning strategies that are specific to Freehand drawing leading to Sketch-specific models that can extract sketch properties and are designed to overcome challenges within the domain. In this part of the research, we will analyze some techniques that have been used for sketch recognition, classification, and representation.

As compared to natural photos that are pixel perfect, hand sketches are highly abstract. A major challenge in coming up with a representation of free-hand sketches to model sketch inputs is to look out for richer representations that are based on efficient storage and at the same time allow for faster calculations. Some popular techniques for sketch representation include storing freehand sketches as a sparse matrix or in the form of strokes or pen coordinates represented in Euclidean space. This representation is particularly useful when considering sketches as time series. Sketches may also be represented as Static pixel space if they’re being rendered as images. Topologically, freehand sketches can also be stored as connected graphs where each stroke may be represented in the form of an edge. Such a representation would also complement the abstract nature of sketches themselves and is expected to be more flexible as compared to the others. Among various other state of art architectures [15] proposed a “SketchRNN” which is a Recurrent neural network-based deep Variational AutoEncoder (VAE) capable of generating sketches in a stroke by stroke manner. This sequence-to-sequence variational encoder has inspired various researchers to use stroke sequences as input to many recent developments in sketch-based models. Due to the efficiency of the stroke-based representation, our application also aims to store user sketches in the form of strokes represented by horizontal and vertical coordinates with respect to time. Figure 2.3 below shows a similar representation of sketches in the QuickDraw dataset.

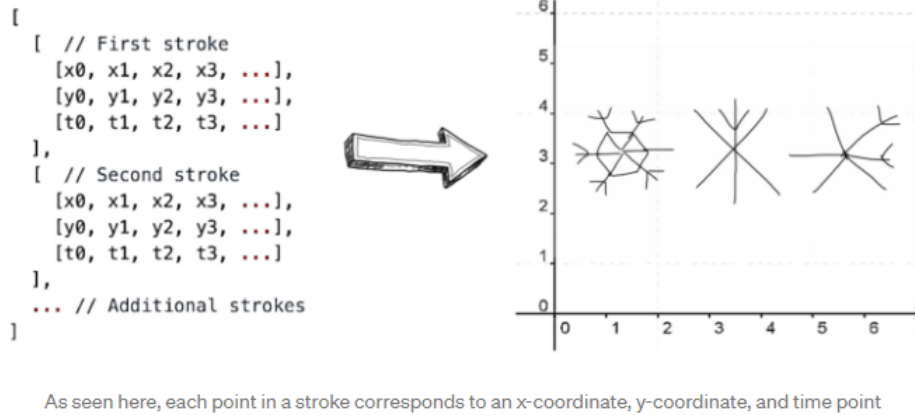


Figure 2.3: Stroke representation of sketches in the QuickDraw dataset

Significant contributions have been made to develop datasets for drawings and sketches. [13] is a Google dataset with a collection of 50 million drawings, divided into 345 categories, collected from the users of the game Quick, Draw!. In contrast with most of the existing image datasets, in the Quick, Draw! Dataset, drawings are stored as time series of pencil positions instead of a bitmap matrix composed of pixels. This aspect makes this dataset the largest doodle dataset available at the time. However, Due to the size of this dataset and the nature of its source, there is a scarce of information about the quality of the drawings contained. [29] is another dataset that is the first large-scale collection of sketch-photo pairs. This sketchy database contains sketches by crowd workers of particular photographic objects sampled from 125 categories and acquire 75,471 sketches of 12,500 objects. The Sketchy database gives fine-grained associations between particular photos and sketches and is used to train cross-domain convolutional networks which embed sketches and photographs in a common feature space.

We can now consider the existing literature and existing datasets related to the classification of sketches concerning the challenges involved. Challenges within sketch classification generally arise due to a great degree of intra and inter-class variations and similarities in hand-drawn sketches. [33] in their survey of freehand sketching categorize the task of Sketch Recognition into online and offline systems wherein offline systems, the model takes an entire sketch as input and returns the predicted class label, while online models predict the class label using sketch strokes in the duration when a sketch is being drawn. As online methods are dynamic, they have proved to be useful for interactive real-time applications. Our application also aims

to employ online sketch classification models to be able to provide real-time feedback to the user. A sketch recognition network proposed by [12] utilizes the sequential process of sketching for training and plots each training sketch as a continuous sequence of cumulative stroke pictures. Then the deep features extracted from the corresponding AlexNet [18] are preprocessed and sent into a Gated Recurrent Unit (GRU) [23] network in sequence. This network generally works for offline methods but has also proved to work well in online recognition mode due to the involvement of the intermediate status of the sketches. [14] discusses traditional Machine learning approaches- including KNN, RNN, SVM, and MLP to recognize hand-drawn images and is particularly focuses on basic shapes like squares, triangle, and oval. [14] gives a performance analysis of traditional techniques as compared to deep learning methods including CNN, LSTM, and WaveNet networks where deep learning models outperform the traditional approaches alongside ensuring computational efficiency.

For automated scoring, [30] proposed Deep convolutional neural networks or ConvNets called DeepSketch that uses a modified structure of AlexNet. The architecture in [30] is specifically designed to extract features from different layers of DNN which leaves room for different facets of similarity. Similarly, [19] employs a deep learning methodology that processes two sketch patches together to generate a similarity score through a semi Siamese network.

Our literature review indicates limited research in the scoring of sketches represented as strokes and a lack of automatic scoring for drawing in general.

2.2.1 Literature Researched

The table on the following pages outlines important details acquired from the literature researched.

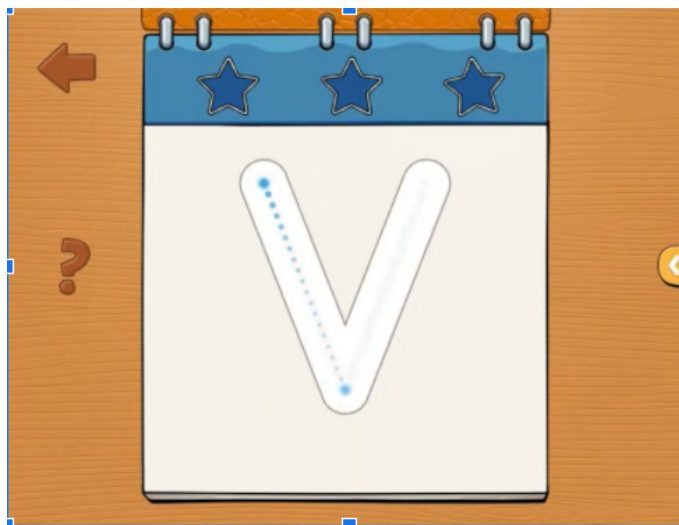
Paper	Dataset	Technique/Architecture	Year	Accuracy
Analysis and Recognition of Hand-Drawn Images with Effective Data Handling	QuickDraw	A comparison of Nearest Neighbor (K-NN), Random Forest Classifier (RFC), Support Vector Classifier (SVC), and Multi-Layer Perceptron model (MLP) was provided against CNN, LSTM, and WaveNet Network.	2019	Max: 82%
Sketch Image Recognition Using Deep Features	HDFS	VGGS Model's pre-trained sketches were preprocessed using MatConvNet library and fed into a semi-siamese network for deep recognition of human sketches.	2016	-
DeepSketch: Deep CNN for Sketch recognition and similarity search	TU-Berlin	Uses CNN for classification, feature extraction and evaluates a similarity index through application of K-Nearest Neighbour.	2015	75.42%
Sketch-a-Net: A deep neural network that beats humans	TU-Berlin	A picture with stroke information is provided as an input to a 5-layered CNN with 8.5M parameters for sketch recognition.	2015	74.90%
Free-Hand Sketch Recognition Classification	Name not specified	PNG files are used for classifying hand drawn sketches through different depths(maximum 25) of the same CNN architecture.	2016	65%

2.3 Related Existing Solutions

We now shift our attention towards existing applications and solutions. This section provides a general overview of the applications which exist for our problem. We will first survey the applications available internationally for other languages, and

then focus our attention towards similar applications in Urdu. We will then analyze applications for drawing and coloring.

There are numerous applications which exist internationally in this domain, covering all of them here is not possible. We will focus and overview three applications, namely, WetDryTry¹, iTrace² and Kaligo³.



captionUI of iTrace

Kaligo, part of the IntuiScript, which was discussed in detail earlier, appears to be the most sophisticated application. It has an English and a French version and provides extensive support to educationalists. The application provides an interface for the educationalists to prepare exercises and a much more sophisticated feedback mechanism. The feedback score is communicated visually to the learner, while also stored to be visualized later by educationalists. The application also supports cursive characters, which is extremely unique in this application. The UI of the application appears to be very carefully crafted to provide an extensive learning experience. However, other than the feedback being conveyed visually to learner and educationalist, it does not provide a personalized learning experience, i.e. focusing on words and alphabets which the learner appears to be weak at.

We now focus our attention towards similar solutions which exist for the Urdu language. Since the applications which exist for Urdu are quite similar to each other,

¹ The overview of the application can be found here. Demo can be found here

² Application can be found here. Demo can be found here

³ Application can be found here. Demo can be found here

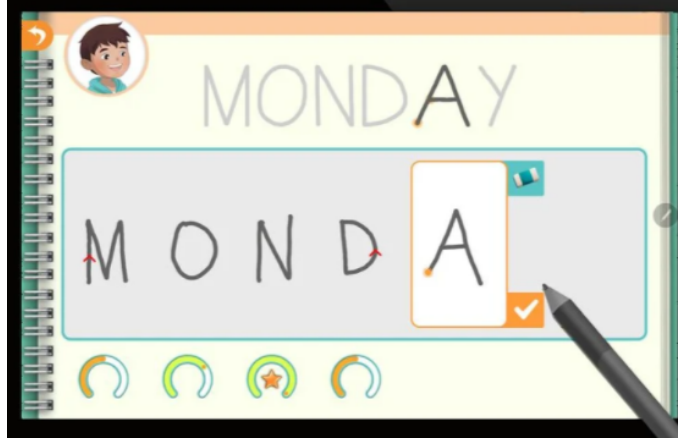


Figure 2.4: UI of Kaligo Application

except for the difference in UI, we will provide a general overview of the applications rather than providing a detailed overview for each application.

There exist few applications for this problem in Urdu⁴⁵⁶ however these applications appear to not have proper documentation hence we will rely on their descriptions on the app store, for their overview. The applications are mostly focused on alphabets tracing with interactive UI. While some applications incorporate visual cues for the learners, the applications generally lack any particular mechanism for feedback. Furthermore, the applications only support Urdu alphabets for tracing and do not incorporate Urdu words in any way. The applications though, try to replicate Urdu writing books by making associations between Urdu alphabets and objects in different ways.

Lastly, we now focus towards applications that are aimed to teach children how to draw and color. A quick survey results in numerous applications but most of these applications are focused on providing a sketchbook-like environment for drawing and coloring⁷, but not actually teaching how to draw. Some applications though are focused on teaching, and providing a learning experience, namely ‘Draw It’⁸, ‘Learn

⁴ Application can be found here

⁵ Application can be found here

⁶ Application can be found here

⁷ <https://sketchbook.com/>

⁸ Application can be found here

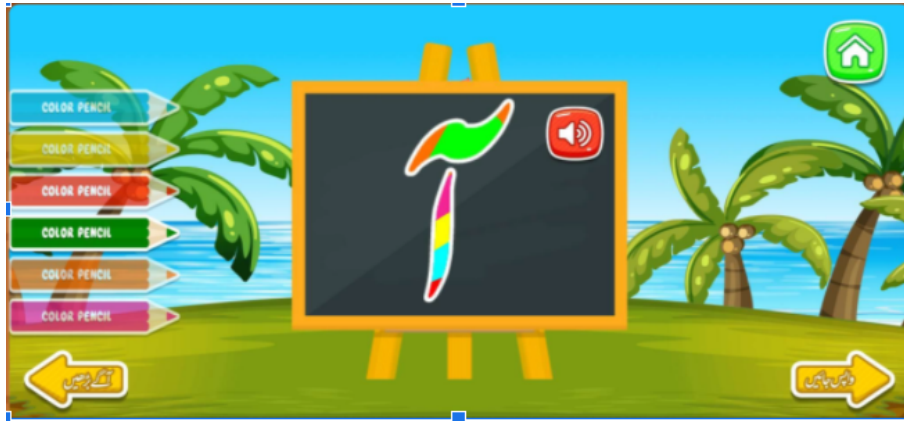


Figure 2.5: UI of Kids Urdu Learning App - Alphabets Learning App

Drawing'⁹,and 'How to Draw – Easy Lessons'¹⁰.

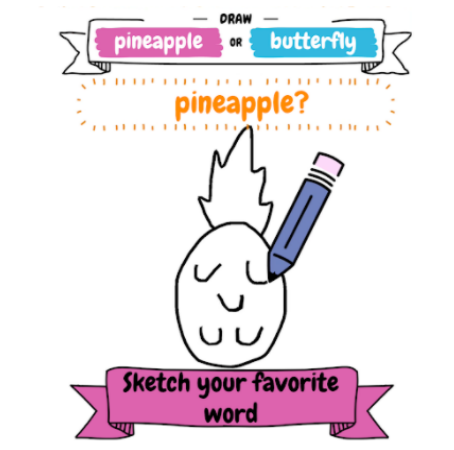


Figure 2.6: UI of Draw it - Learning App

⁹Application can be found here

¹⁰Application can be found here

3. Software Requirement Specification (SRS)

This chapter provides detailed specifications of the developed system.

3.1 System Overview

The application is divided into two main modules namely Urdu, Drawing. Each module contains its respective exercises that are personalized for each user. The application also visualizes a learner's overall progress and this progress can be accessed through the dashboard.

3.1.1 Module: Teaching Urdu

The module would teach to understand and write Urdu letters and basic Urdu words, would enable learners to map Urdu words with their pictorial meanings, and teach how Urdu letters and words are pronounced.

1. Sub-module: Writing Urdu Letters

This sub-module would teach to write and identify Urdu letters accurately. Learners would be taught to write Urdu letters by following a Urdu text book (Qaidah) approach. Learner will be taught to write Urdu letters on the blank canvas like writing on an empty page, and the attempt will be scored. With each exercise, an image of sample Urdu letter would also appear on the screen which the learner can replicate. Moreover, an animated object corresponding to the letter would also be shown.

2. Sub-module: Writing Urdu Words

This sub-module would teach learners to join Urdu letters to make words. Similar to Urdu letters, learners will be taught to write Urdu words on the blank

canvas like writing on an empty page, and the attempt will be scored. Moreover, an image of sample Urdu word and an animated object corresponding to the word would appear on the screen.

3. **Sub-module: Recognizing through flash-cards**

The sub-module would help create an identification of Urdu words and objects for learners using flashcards. The sub-module would have 2 ways of representation:

(a) **Recognizing Urdu words through flashcard images**

The app would show an image (like flash card) and a few Urdu words. The learner would be expected to select the right word which represents the flash-card image.

(b) **Recognizing images through flashcard Urdu Words**

The app would show an Urdu word and a few images. The learner would be expected to select the correct image which represents the word.

4. **Urdu Personalization and Feedback:**

(a) **Writing Urdu**

With every try at writing a Urdu letter or word, the app would use Machine-learning and statistical models and stroke analysis to analyze and score the accuracy of the learner in writing it. App would then provide intuitive real-time feedback to the learner. The scoring mechanism would also take into account the level the learner is at. For personalization, the app would analyze and keep a record of the type of letters and words which the learner finds difficult to write and based on the analysis the app would subsequently provide similar practice exercises to the learner. Types of letters types can be categorized in terms of their writing style, number strokes required to write a letter, or their visual or auditory similarity with other letters.

(b) **Recognizing through flashcards**

The application would keep a record of the objects and words which the learner struggled in identifying and would then incorporate different images of the same flashcards in subsequent exercises.

3.1.2 Module: Drawing

The module would teach to draw from scratch and will score attempt of the sketch. Drawing module would be to teach to draw very basic structures of shapes and every day structures with basic guidance and sample drawings. Drawing can for example be shapes, and structures that are made of shapes like tree, ball, ants.

Personalization and feedback of Drawing

The app would analyze and store the performance of the learner. With each try of drawing, the app would analyze the drawing. The application would provide the learner more exercises that are similar to the ones the learner is facing difficulty in.

3.1.3 Performance Reporting and Visualization

Guardians can use dashboard to view progress of any child by choosing a profile. The dashboard would include factors such as time spent by learner, feedback received on each exercise and improvement demonstrated by learner over time.

3.2 Functional Requirements

This section describes each function/feature provided by our system. These functions are logically grouped into modules based on their purpose/users/mode of operations etc (as per our system).

1. Sign up
Sign up allows a first-time user to create an account with a unique email ID. This ID (alongside a password) will be used by them to later login to the system and consequently be able to use the application. At the time of sign up the user will have to enter the first Name and last Name, email address and password.
2. Login in
A one-time login. Users who have already signed up can login into the app using their Email ID and password.
3. Manage Profile
A profile is specific to one individual learner. Each learner will use his profile to attempt exercises. The parent or guardian is expected to manage all profiles created through their credentials.

- **Add Profile**
User can create a new profile in the system for a new learner. To create a profile the user will have to enter a child's username, age, gender and can also select a profile avatar.
- **Edit Profile**
Information about the child's profile can be edited. User can edit child's username, name, age, gender and/or change the profile avatar.
- **Delete Profile**
The user can delete any profile from the system. The performance a child made under this profile and all other data associated with the profile will be lost as a result.

4. Assessment

- **Scoring**
Machine Learning or Statistical Model(s) together with a combination of several factors including time taken, correctness, stroke estimation will generate a score. If the score is below a threshold, the child will not be able to move forward to the next level.
- **Identifying Common Mistakes**
The Machine Learning or Statistical Model(s) will be used to identify the common points (for example curves, straight lines, dots on the alphabets) and the area in which the child makes more mistakes will occur more frequently in future exercises.

5. Exercises

- **Exercise Generation**
The application consists of a model that is able to utilize the feedback score to produce the next set of practice problems from the set of exercises in the system for a specific user. These exercises are generated based on a learner's needs as evaluated from his score and previous records.
- **Timer**
A timer will efficiently keep track of a learner's activity and the time taken to complete the assigned tasks. The timer keeps a record of time taken per exercise and the time taken per module by a child.
- **Speech/Audio**
The user can listen to the speech/audio of the words/letters/colors/objects that will appear on the screen. This shall help the user to know how

the words/letters/colors/objects that will appear on the screen are pronounced. The speech will help the user to build a mapping and be able to speak, understand and comprehend similar words whenever encountered.

- **Tracing**
There will be a dotted patterns of words/drawings and the learner will have to trace on the screen following the pattern. This will help the child in learning to write/draw.
- **Database**
All exercises, feedback score, history, time taken and speech/audio will be stored in the database and will be accessed when needed.

6. Settings

- **Adjust Volume**
Through a scroll bar the user can adjust the audio level in the application.
- **Reset Password**
The user can reset his/her login password for the application. To do so the user will have to provide the current password.

7. Dashboard

- **Exercise History**
The progress history of each exercise can be viewed from the dashboard. The progress history will show the score of the exercise, the number of attempts taken till the exercise was passed, time spent on the exercises of the module and the feedback score of the exercises.
- **Progress Visualization**
The user can see the overall performance, progress, feedback score and rate of improvement with respect to time taken through different visualizations.

3.3 Non-functional Requirements

3.3.1 Performance

The user data increases over time, and the application stores data in the cloud in regular intervals, therefore the system requires good processing power and better hardware capabilities. Cloud storage also requires good internet connection. The

system should be able to handle database lock issues alongside concurrent use and access of the database. For all UI elements that do not involve internet connection, the system shall respond to user input immediately with no noticeable lag. Fields and actions that require processing in the database may take around 3-4 seconds. Exercise generation will take around 5 seconds. Feedback generation may take around 5-6 seconds.

3.3.2 Reliability

The system can be relied upon as per the user's need - to generate feedback, create profiles, generate exercises and manage history without failure. The data will be stored in a database on the application which will be synced to the cloud. In case the application malfunctions, the data maybe recovered from the cloud.

3.3.3 Availability

The application is built over an android platform and can be downloaded on a tablet. The user can then sign up or log in from his account to continue using the application at any time.

3.3.4 Maintainability

The system can be modified, updated and bugs can be fixed whenever required. New features may also be introduced later on.

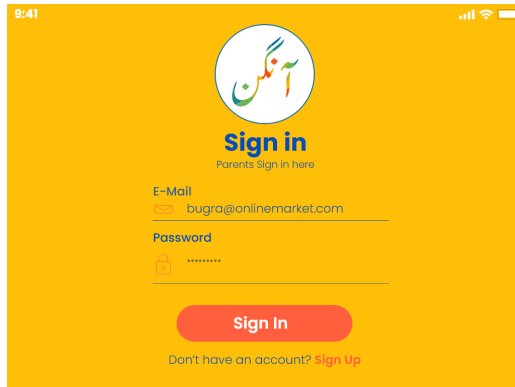
3.3.5 Scalability

Bugs may be fixed and new features may be added to the system from time to time. New exercises may also be added to the database.

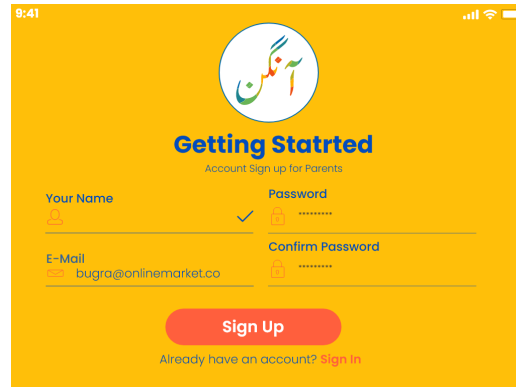
3.4 External Interfaces

3.4.1 User Interfaces

The application has several key-screens that are described in this section.



(a) Sign In



(b) Sign Up

9:41

9:41

Select your profile



(c) Profile Selection

Figure 3.1: Introductory Screens

As the application opens, the user is presented with the screen shown in figure 3.1a. The user in this case is the parent/guardian. If the user already has an account, they can sign-in through this screen or else they will be re-directed to 3.1b screen through the 'Sign Up' button. After both signing up and signing in, the user is re-directed to 3.1c where they can select the profile child who is going to use the application for attempting exercises.

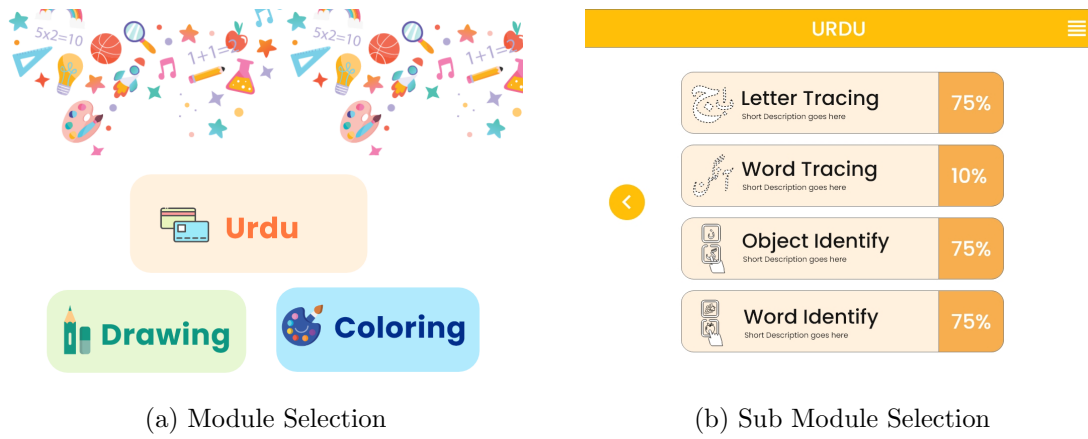


Figure 3.2: Module Selection

After a profile is selected, the user now is the child. For all successive screens, the user will be the learner/child. 3.2a is the first screen that appears after profile selection. Here the user has to select which module to play. Once the module is selected, the user can then select the sub-module from 3.2b.

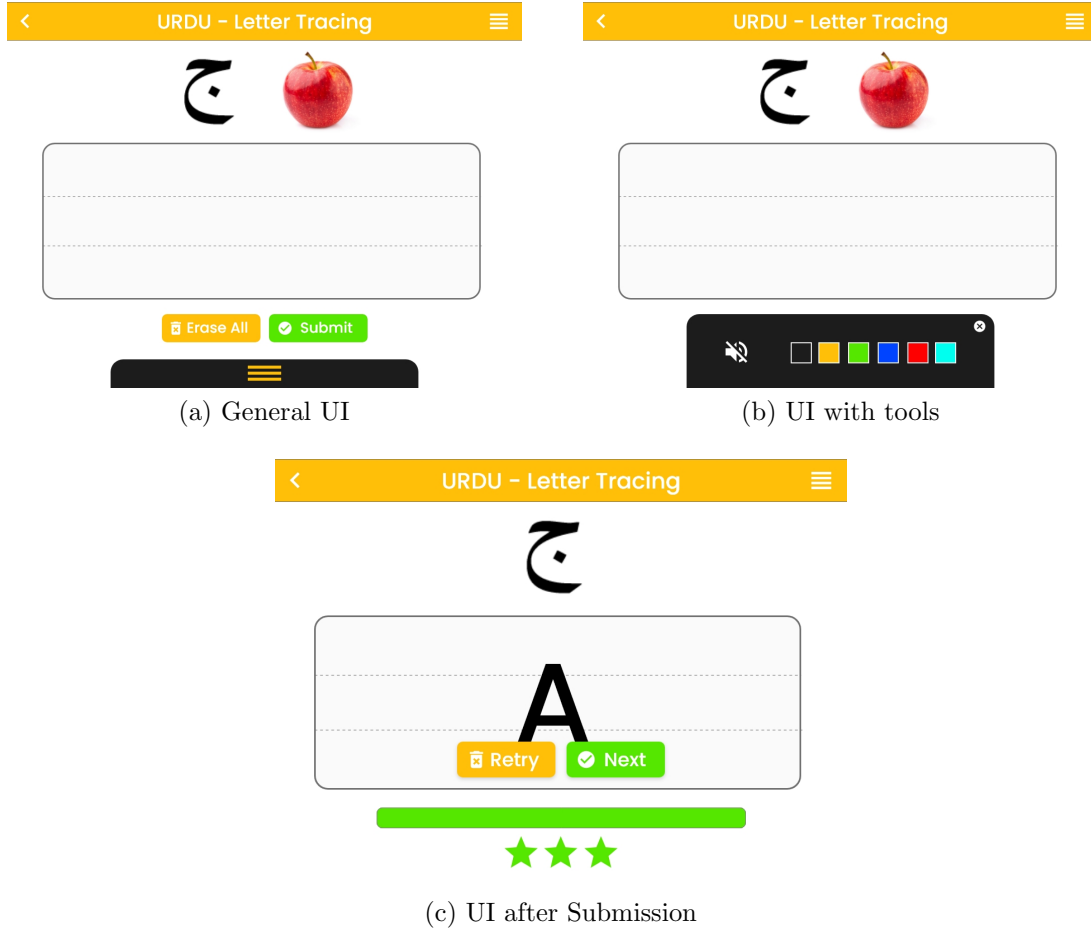


Figure 3.3: Urdu Module - UI

3.3a illustrates the UI inside the Urdu module. We have two buttons:

1. the 'Erase All' button that removes everything that the user has written.
2. the 'Submit' button that submits the user's writing for feedback.

The tile at the bottom, when clicked, provides options to customize the experience to the user. 3.3b shows the various customization options available inside the options tool box, while 3.3c shows how the feedback is communicated to the user. The user will also have options to retry or move to the next exercise depending on his performance.

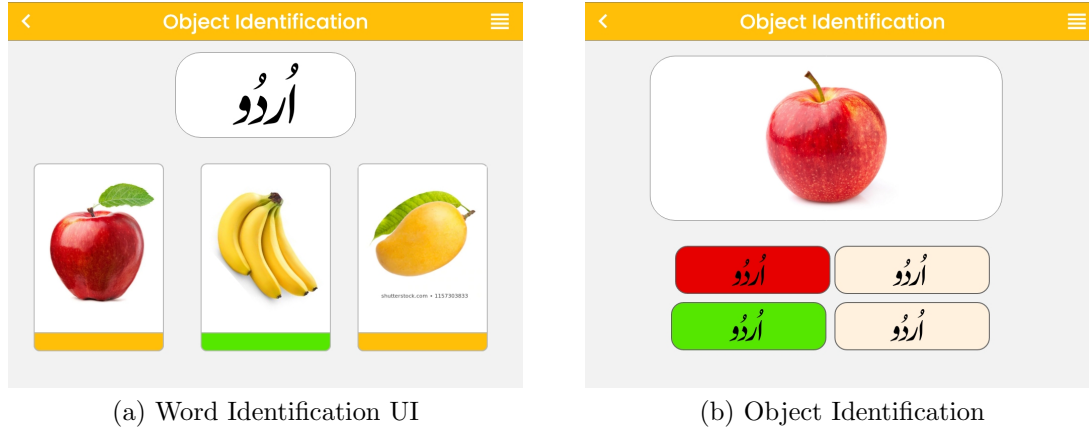


Figure 3.4: Remaining modules for Urdu

similarly, 3.4a shows the UI for the word identification module. The user has to correctly identify the object corresponding to the given word. If chosen correctly, the tile turns greens. If not, it turns red. 3.4b shows the reverse case where the user has to identify the word associated with the object.

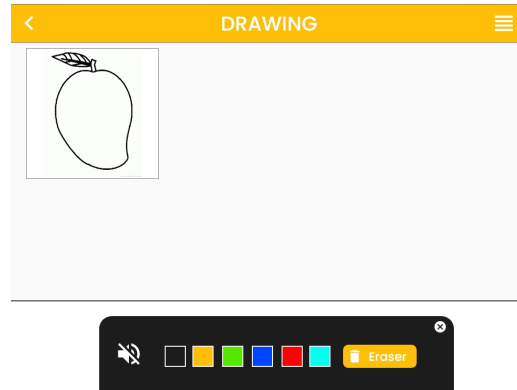


Figure 3.5: Drawing module

3.5 shows the interface for drawing. The interface is almost same as the interface for Urdu, with the addition of a button for eraser.

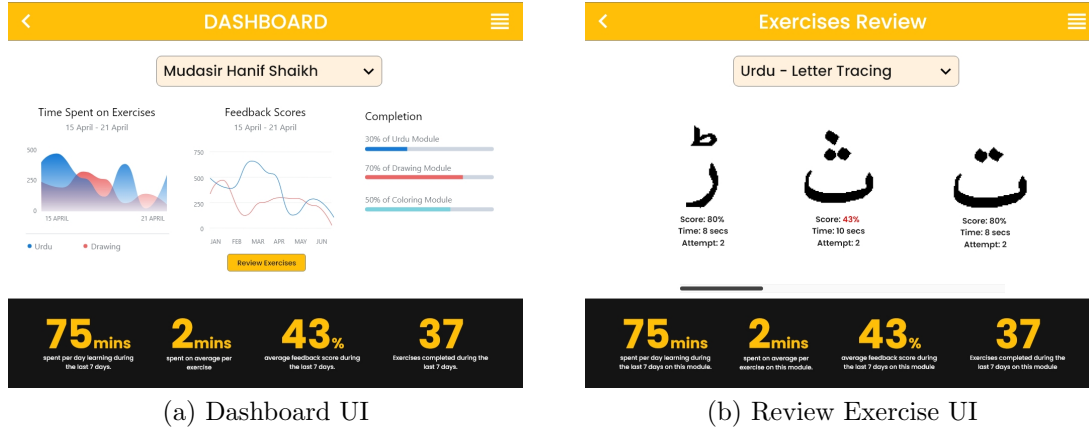


Figure 3.6: Performance Visualization Screens

3.6a and 3.6b shows the interface available to the parents to visualize their child’s performance. They can select the name of the child whose performance they want to visualize, and then the dashboard will display the progress of the child. The dashboard comprises of various graphs and summary statistics, and a button of to review exercises in detail. The ‘review exercises’ button takes the user to 3.6b through which each individual exercise can be viewed along with the summary statistics for individual module.

3.4.2 Application Program Interface (API)

Our application involves exploiting a bunch of useful modules in python, primarily for statistical and deep learning analysis. We will use the TensorFlow framework for our deep learning models. However, this will not constitute the Application Program Interface of our app, as we will build our own application program interface in the form of a REST API using Django REST framework. All our models will be deployed in the form of endpoints on the cloud, and our application front-end will interact with these models through our REST API using GET/POST methods.

3.5 Use Cases

Figure 3.7 shows use-cases of the entire application, while figure 3.8 shows use-cases specific to the Urdu module.

There are 2 users of the application: Guardian and learners/children. Guardians can create their profile on the application and then add profiles of the children to maintain the history and performance of each child separately. Guardians can manage these profiles and can view the performance of each child via the dashboard.

Children can select and visit a learning module. Available modules are Urdu and drawing. Within the Urdu module, a learner can choose from available sub-modules which are Urdu letters, Urdu words, identifying objects, and identifying words. The system generates personalized and relevant exercises for the user and maintains a record of the time taken by a learner to attempt an exercise. As the name suggests, a learner learns to draw in the drawing module. In the Urdu letters sub-module, a learner can trace and write Urdu letters, and in the Urdu words sub-module a learner can trace and write Urdu words. In "identifying objects" sub-module, a child can select objects, and in the identifying words sub-module, a child can select words.

In response to the attempted exercise, the system generates a score which is used to generate feedback.

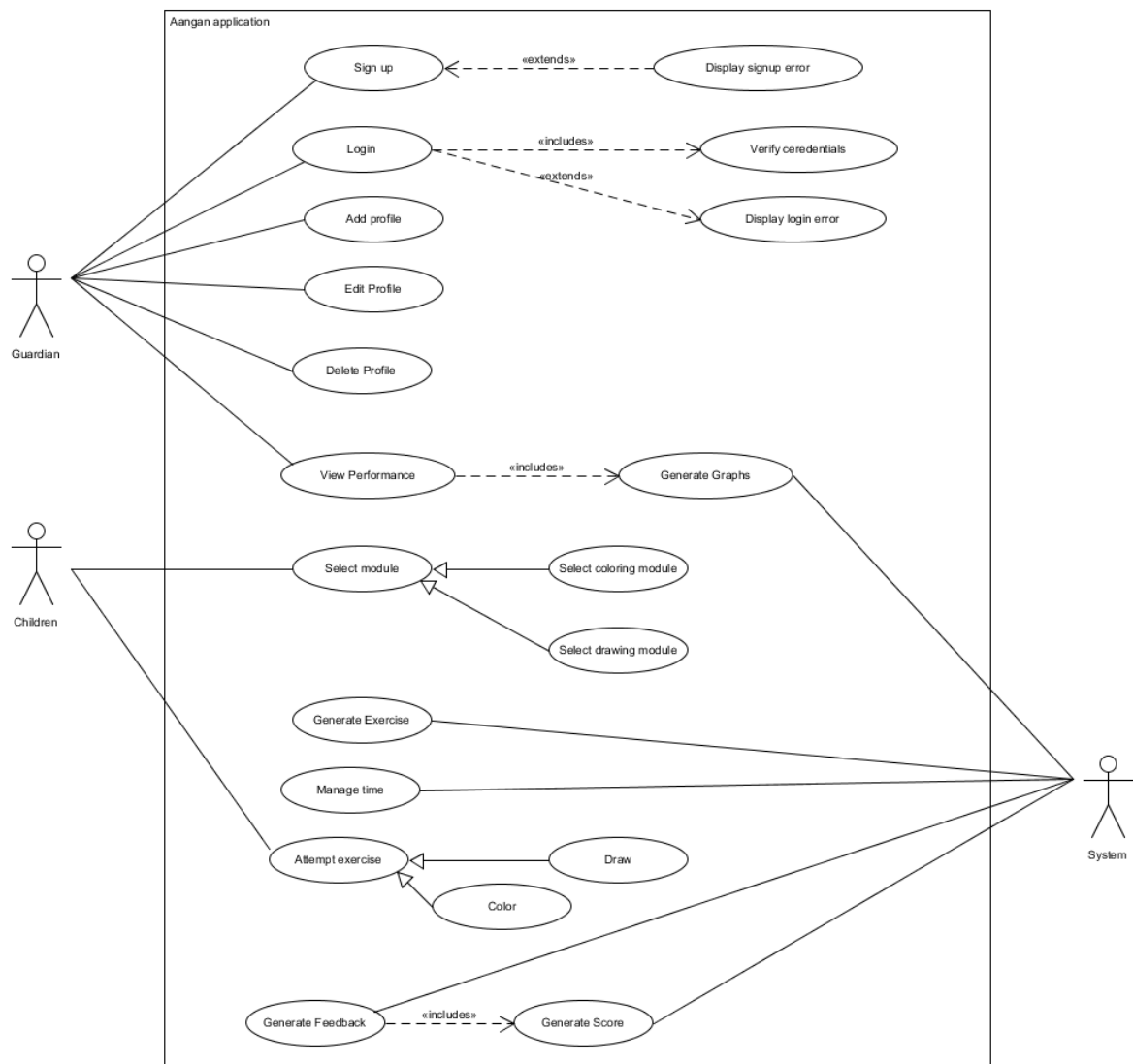


Figure 3.7: Usecase diagram of the application

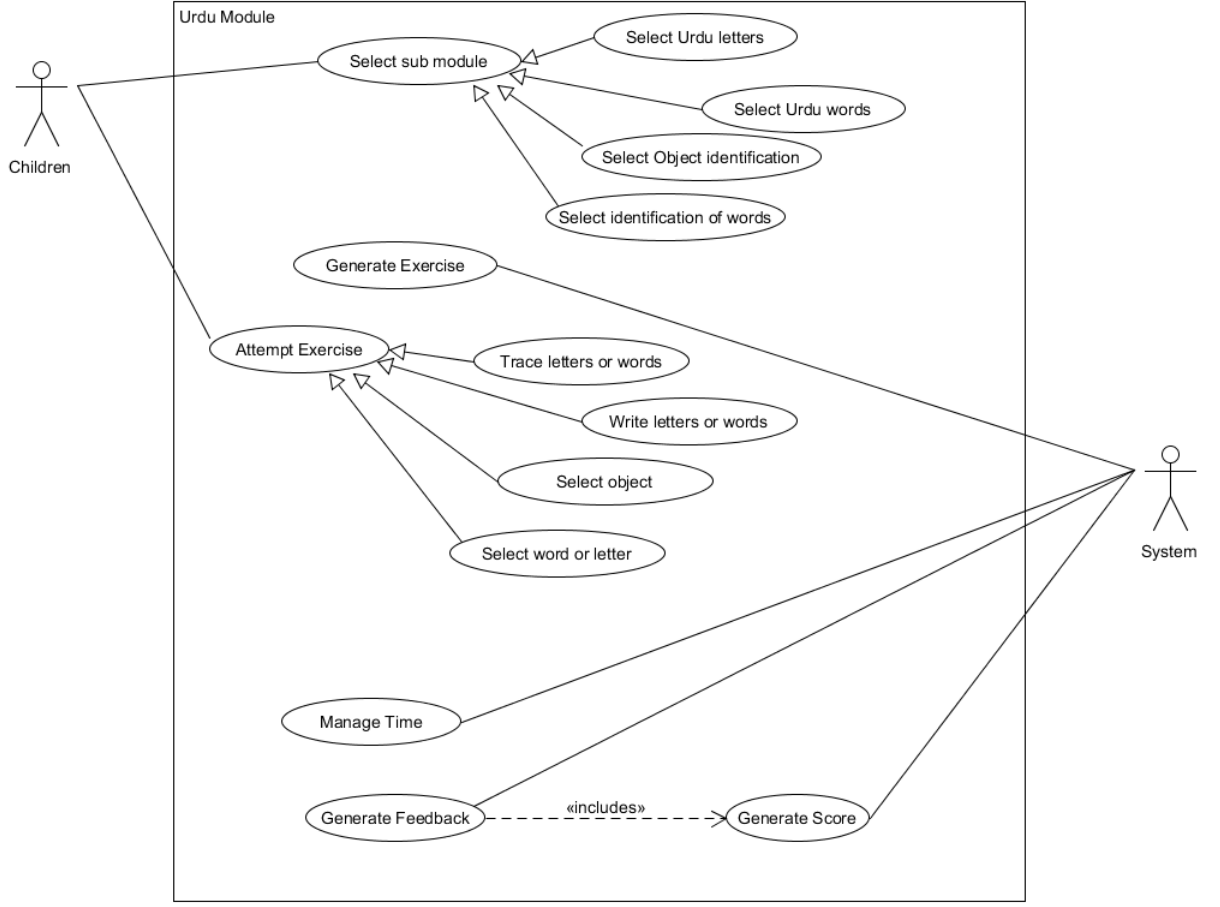


Figure 3.8: Usecase diagram of Urdu module

3.6 Datasets

This section describes the specific dataset(s) used to build our system. An appropriate snapshot of the dataset(s) is also included. Further details, when needed, are present in the appendix.

1. QuickDraw dataset[28]

The dataset stores drawings as time-series of pencil positions instead of bitmap matrix composed of pixels. This makes it the largest doodle dataset available right now. The feature also makes it an ideal resource for the our model that

will be responsible for stroke analysis. Furthermore, the dataset shall assist us for better scoring strategies by providing key information regarding stroke.

2. Urdu Handwriting dataset[21]

The dataset contains handwritten Urdu letters/characters written by more than 900 individuals. This dataset will be particularly useful for the Urdu module to determine the correctness of a traced alphabet by a learner through a Urdu text-recognition model.

3. Offline Urdu Synthetic dataset[32]

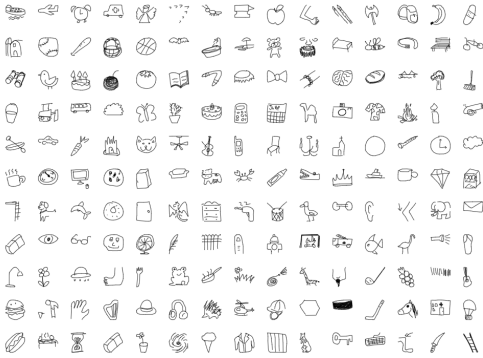
Urdu scripts are generally considered complex because of a number of similar characters, position, curvature of characters and context sensitivity. The cursive script particularly contributes to making urdu a favourable language for research in OCR. This dataset contains machine rendered text images with little noise specifically designed for OCR. This shall help us to determine the correctness and score estimation of the user for any particular character.

4. Dataset for Urdu OCR[10]

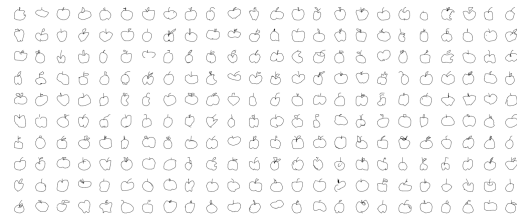
The dataset contains 800 images of each of 38 urdu handwritten characters and 10 numerals. This dataset particularly focuses on characters with similar shapes than the number of strokes and can facilitate the scoring model by adding better measures for estimating a correctness score for similar letters.

5. Arabic Synthetic handwriting dataset[20]

The dataset contains 16,800 characters with contributions from 60 participants. It has taken into consideration issues such as shape similarity, dots and positioning and can be of great help with evaluating a learner's score and writing patterns.



(a) Quick Draw Dataset



(b) Apples from Quick Draw

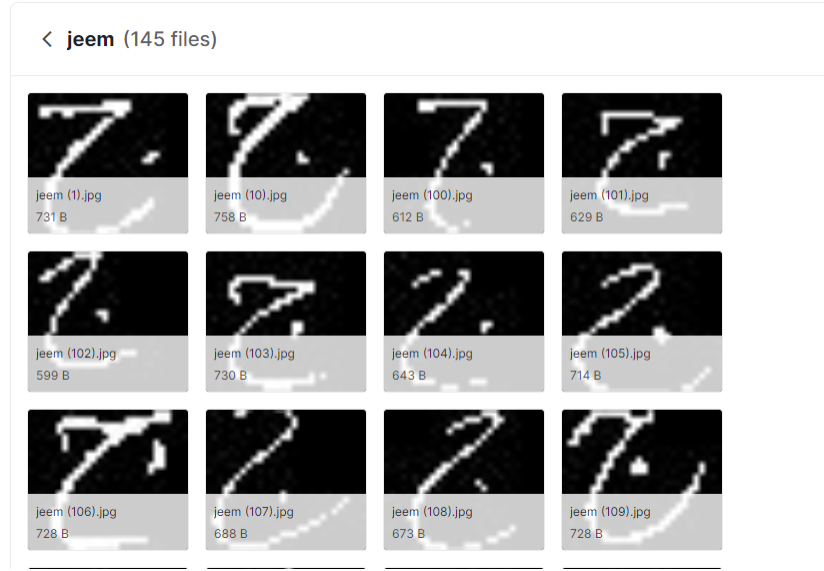


Figure 3.9: UHaT: Urdu handwritten text dataset

3.7 System Diagram

This diagram gives a high-level view of the different components of our system and the interactions between them. Each component and the particular tools/technologies/libraries used to build it are described.

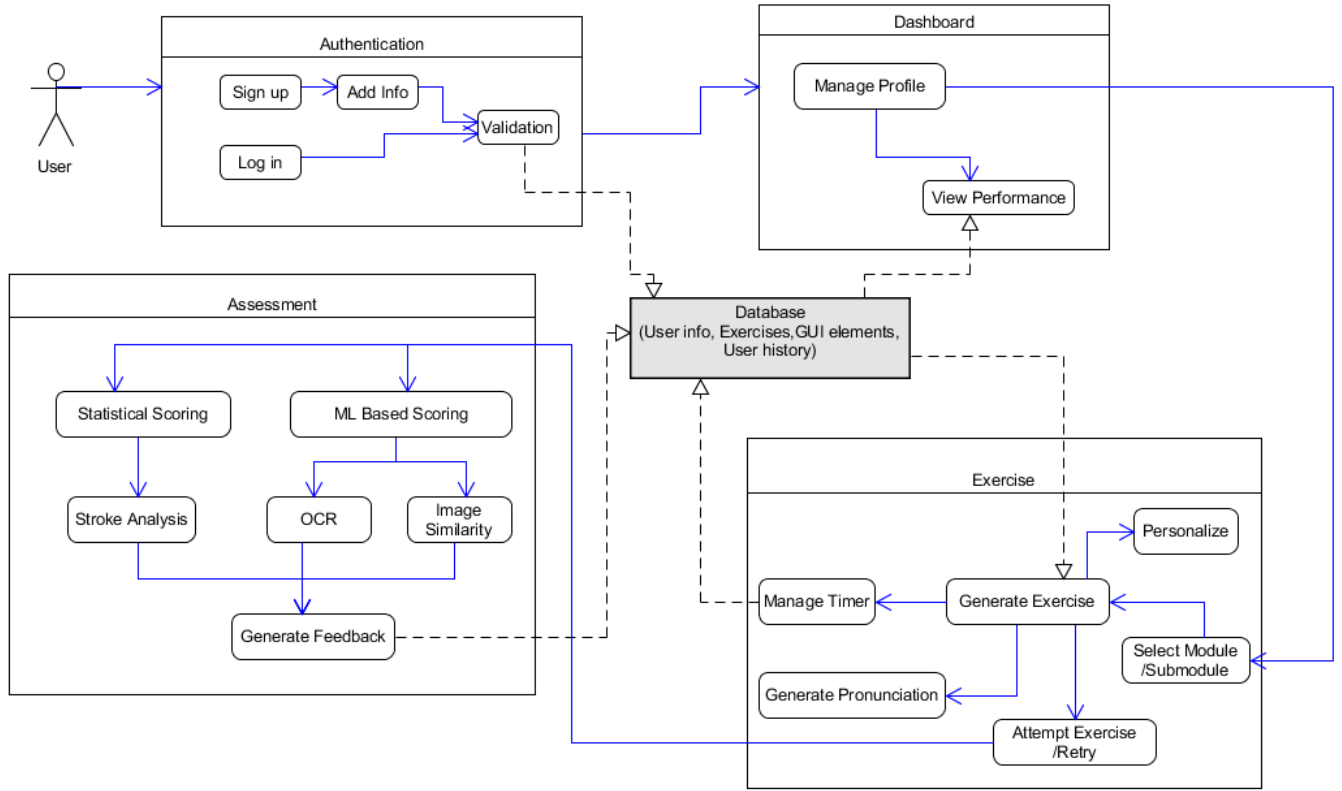


Figure 3.10: System Diagram

The system is based on the following modules:

1. **Authentication** is concerned with validation of user credentials and defines elements used to access the application. The application access for the first time requires an account that can be created using sign-up. The user can then use his credentials to log into the system anytime.
2. **Dashboard** allows parents to add new profiles to the system and edit or delete existing profiles from the system. Every child accessing the application from a single account will have his own profile in the system. The dashboard also allows parents to access history of individual profiles and view performance (score, exercise details, attempts etc) of every child.
3. **Exercise** defines different features associated with attempting exercises on the system. A user can select a module either Urdu and drawing from the

application modules. Each module directs the user to its respective exercises. If a user selects the Urdu module, he is directed to the submodules. The Urdu submodules allow a user to trace letters, trace words, identify objects or identify words. The exercise generator generates and filters exercises from the database for the user to attempt. These exercises are personalized and are therefore different for every user. The exercises also contain an audio that pronounces the word/letter/object being displayed on the screen. The timer begins recording the time once the user starts attempting the exercise.

4. **Assessment** defines methods used to evaluate an attempt by the user. When a user submits an attempt, it is analyzed through different Machine learning and Statistical methods to identify common mistakes, pattern and similarity to the original letter/word/object. These methods include Stroke analysis and OCR among others. Based on this analysis, the application assigns a score to the user that determines whether a user can proceed to the next level. This score is added to the progress of the child and contributes to his overall performance.

4. Software Design Specification (SDS)

This chapter provides important artifacts related to design of our project.

4.1 Software Design

This section presents the UML class diagram and gives a brief description of each class in our system. A high quality image of the class diagram can be found here.

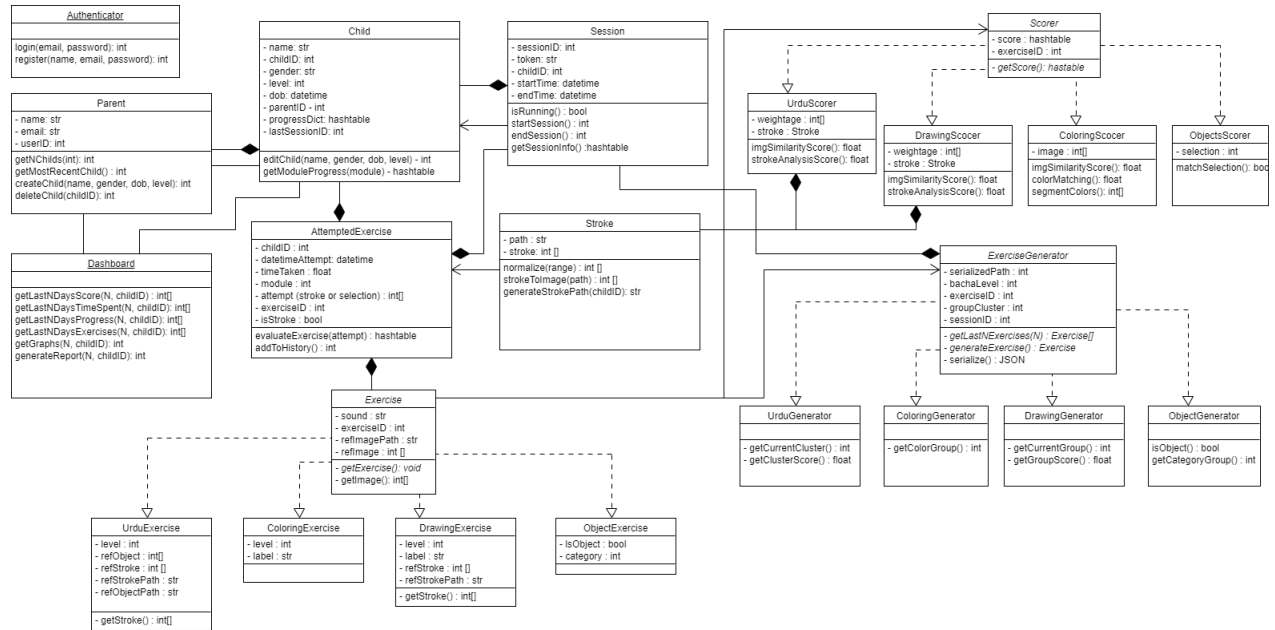


Figure 4.1: UML Class Diagram of the system

The main components of the class diagram are explained below:

Authentication, Session, Parent and Child

- Authentication class is responsible for authenticating the user, that is, the parent and allowing them access to the application.
- Session class is responsible for storing session level information of the user. A session is characterized by what actions a user performs when they start the app. A new session is initiated every time the user opens the app.
- Parent class is responsible for dealing with methods that may relate to a parent's role in the application. This includes for example adding/deleting child as well as showing the parent's current child(s) in the application once they are authenticated.
- The child class deals with the attributes and methods that may relate to the end user, that is, the child's role in the application. This includes for example editing the information of the child or getting the current progress of the child.

Exercise and Stroke

- Exercise class is responsible for getting the exercise from the database and passing it onto the front-end. This class deals with all things related to an exercise that are needed to display it successfully for the end user including the reference images as well as the strokes. The class contains four sub-classes, each corresponding to the four different modules of our application since every module has separate requirements for the exercise.
- A stroke is typically stored in form of a list of $[x, y, t]$ coordinates, where x , y are position and t is time. The Stroke class is responsible for manipulating the stroke data structure that would be used to store the user's attempt of the exercise. We would also be storing the reference Urdu letters and words in form of a stroke to do better analysis.

ExerciseGenerator

- This class is responsible for generating personalized exercises based on user's past progress, and implementing business logic based constraints such as ensuring sequence of Urdu alphabets. The class contains four sub-classes, each corresponding to the four different modules of our application. Each module has a distinct logic for generating the exercise which is implemented in the virtual function *getExercise()*.

AttemptedExercise and Scorer

- As the name suggests, the AttemptedExercise class is responsible for recording and saving the user's attempt. It will also evaluate the exercise by triggering the scoring methods of the scorer class depending on the module which that particular exercise belongs to.
- The Scorer exercise is responsible for evaluating the exercise and generating a score that is to be saved and sent back to the user as feedback. The class contains four sub-classes, each corresponding to the four different modules of our application. Each module has a distinct method for scoring the exercise, with separate models using which the combined is generated. The logic for generating the final score is implemented in the virtual function *getScore()*.

Dashboard

- This is a static class which is responsible for the operations that would be required when displaying the feedback and performance of a child to their parents. This includes, but is not limited to, generating reports and corresponding graphs for each child.

4.2 Data Design

This section presents the structure of our database that caters to persistent data storage in our project. A high quality PDF of the ERD can be found [here](#).

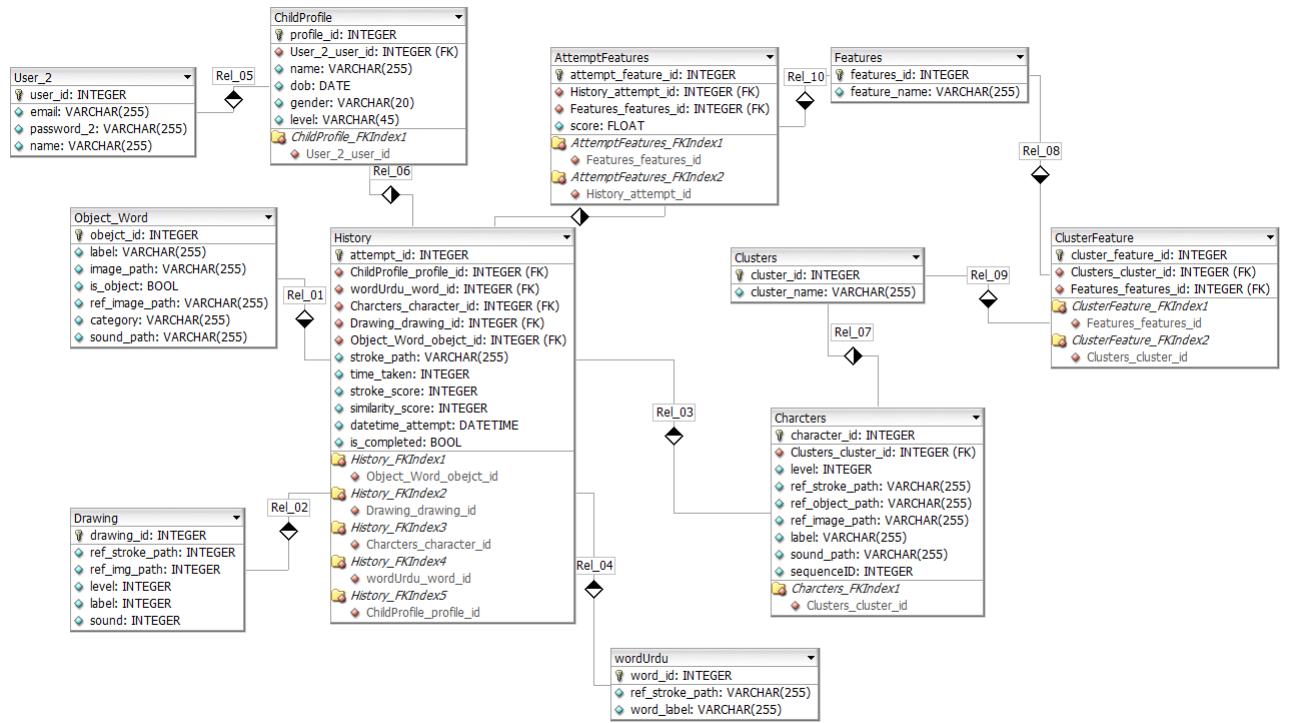


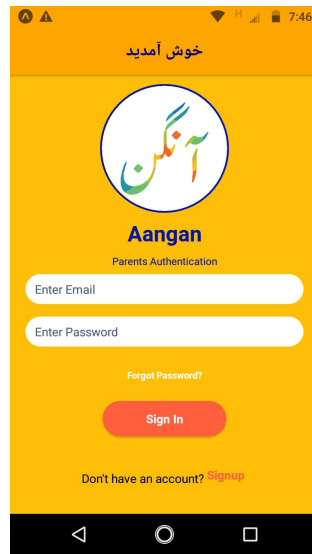
Figure 4.2: Entity Relationship Diagram of the system

5. Application

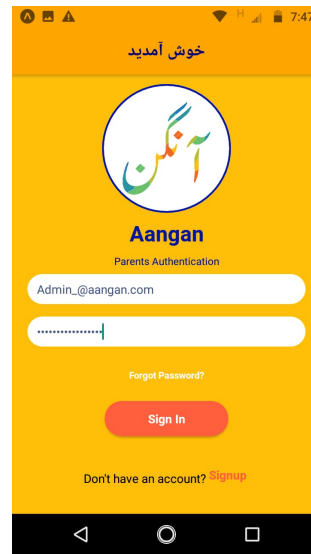
This chapter provides a walk-through of our application and discusses the functionality implemented.

5.1 Register, Login and Main Menus

The first screen that the user sees on opening the application is the Login screen, as shown in figure 5.1a. If the user has already registered on the application and has an account, they can log in by entering email and password as shown in figure 5.1b. The application validates that the email exists in the application's database i.e. the user has an account, the email entered is correct, and the password entered is correct. If the user has not registered, they are required to do so. The user can also reset the password if the user has forgotten the password by using the option shown in figure 5.1a. The user can reset a new password by as shown in figure 5.2



(a) Login



(b) Login with filled entries

Figure 5.1: Logging in to application

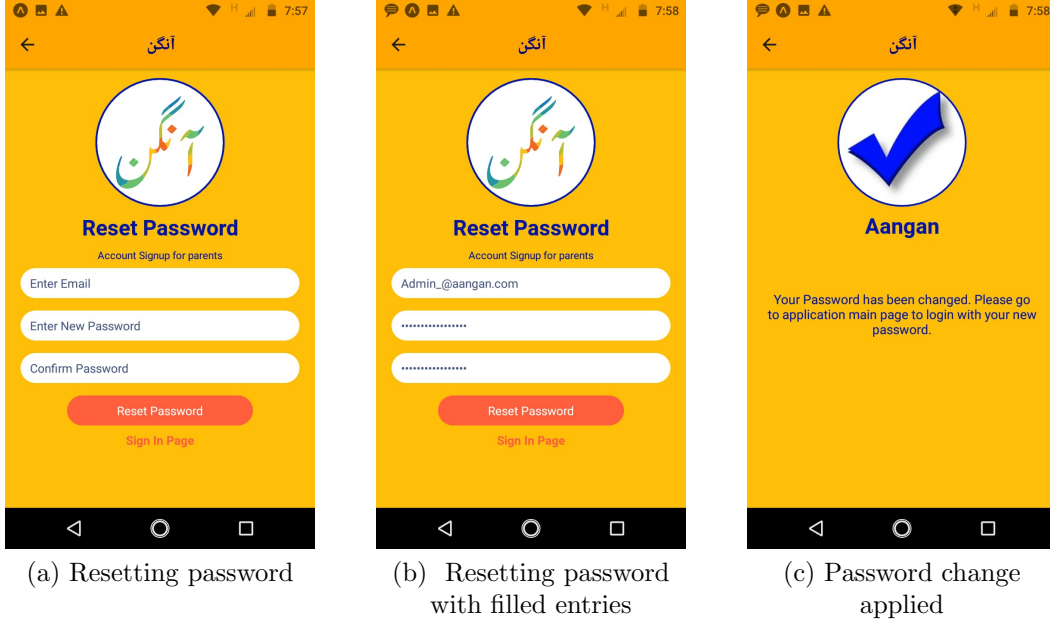


Figure 5.2: Urdu letter tracing

Figure 5.3a shows the Sign-up screen of the application. The user i.e. parents, guardians, or educators can register by providing details such as name, email, and password as shown in figure 5.3b. To help the user avoid the 'cost of a mistyped password', sign-up requires the user to retype the password. Validation is applied on each text entry and does not allow for empty inputs. Following validations were applied on password to ensure a strong and unique password:

- User Attribute Similarity Validator: checks for the similarity between the password and a set of attributes of the user
- User Attribute Similarity Validator: checks for the similarity between the password and a set of attributes of the user
- Minimum Length Validator: checks whether the password meets a minimum length
- Common Password Validator: checks whether the password occurs in a list of common passwords
- Numeric Password Validator: checks whether the password is not entirely numeric

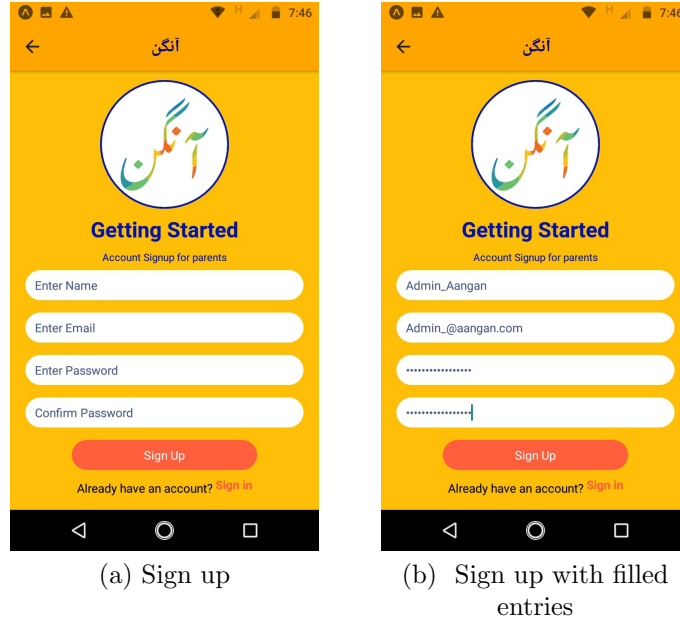
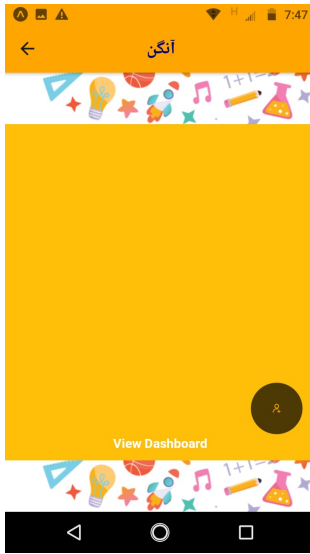
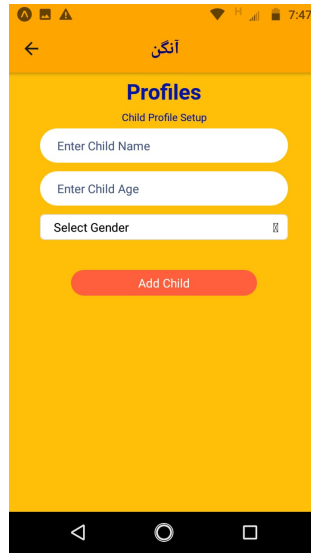


Figure 5.3: Registering to use application

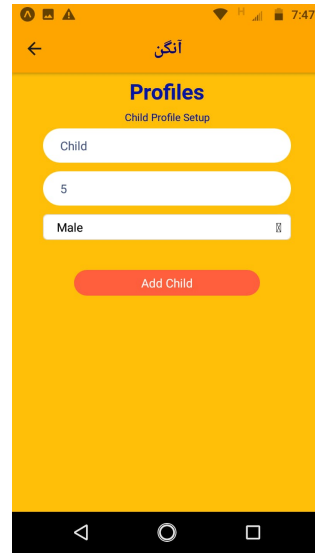
After logging in to the application, the first screen the user sees is to select a child profile. The user i.e. parents, guardians, or educators can use the same account for up to 3 children. If the user is logging in for the first time, there are no child profiles added as shown in figure 5.4a, so the user must add a child profile to proceed. By clicking on the add profile button, the user is directed to add a profile screen as shown in figure 5.4b where the user must add the name, age, and gender of the child as shown in figure 5.4c. Validation is applied on each entry and does not allow for empty inputs and age must be a numeric value. After the child profile has been created as shown in figure 5.5 the user can now select a child profile to visit modules as shown in figure 5.6.



(a) New user to add child profile



(b) Add child profile



(c) Add child profile with filled entries

Figure 5.4: Adding child profile



Figure 5.5: Selecting child profile

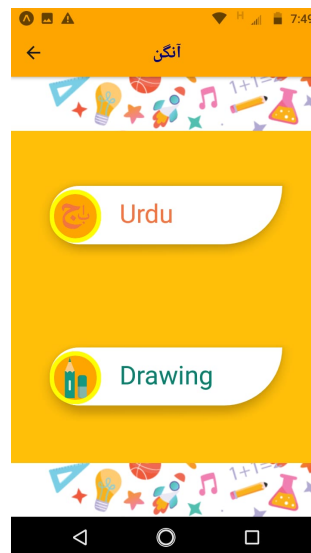


Figure 5.6: Selecting module

5.2 Urdu Module

On selecting Urdu from modules, the learner is directed to the Urdu module screen as shown in figure 5.7. The learner can select a sub-module as desired.

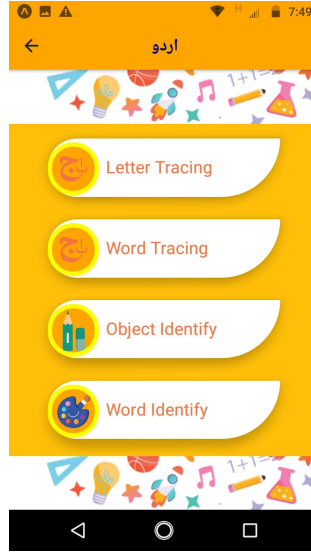
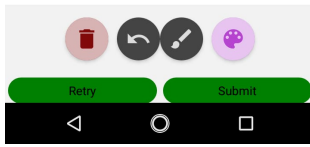
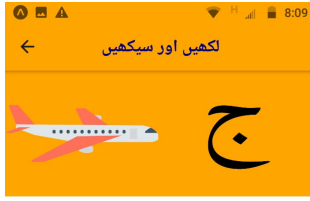


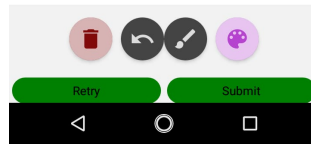
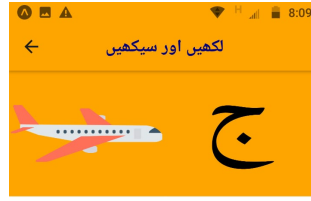
Figure 5.7: Selecting Urdu module

5.2.1 Urdu Letter Writing

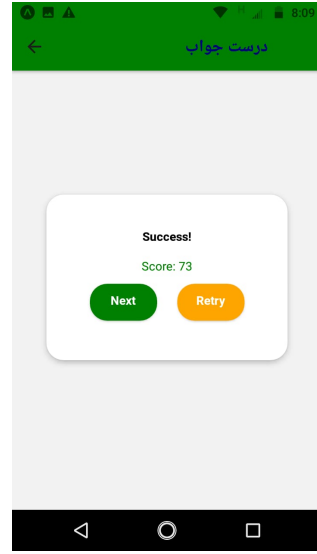
Figure 5.8a shows the screen of an exercise if the learner selects to learn Urdu letters. Just like Urdu qai'dah, the learner can see a sample image of the Urdu letter and a corresponding object that starts with the same letter. The learner can write the letter on the canvas using their device touch screen as shown in figure 5.8b. Canvas allows the learner to undo a stroke, delete the Urdu letter's sketch, change the color of the pen, and change the width of the pen. After writing the letter, the learner can submit the attempt of the exercise. As feedback, the learner immediately receives a score out of 100 on the attempt as shown in figure 5.8c. If the attempt of the learner is not correct as shown in figure 5.9a and the score is less than 50, the learner can not move forward to another exercise and will have to re-attempt exercise to improve as shown in figure 5.9b.



(a) Letter tracing screen



(b) Writing letter on canvas



(c) Feedback on correct writing of letter

Figure 5.8: Urdu letter tracing

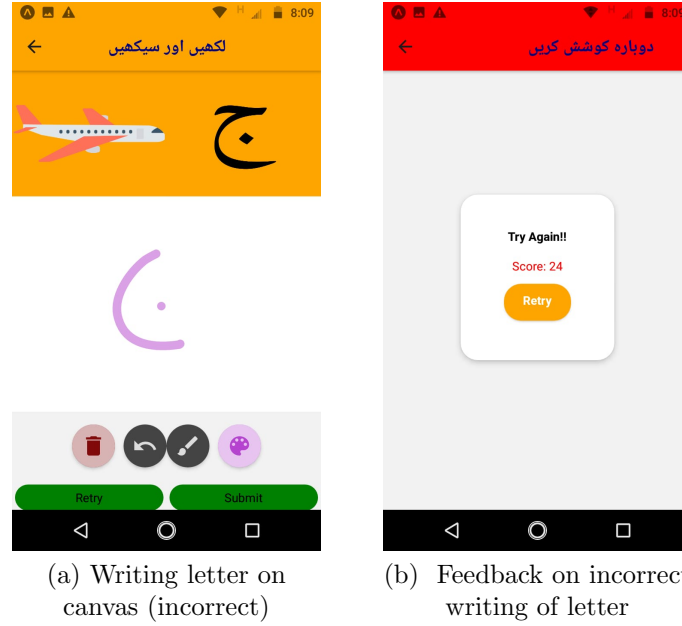


Figure 5.9: Urdu letter tracing for incorrect attempt

5.2.2 Urdu Word Writing

Figure 5.10a shows the screen of an exercise if the learner selects to learn Urdu Words. Just like Urdu qai'dah, the learner can see a sample Urdu word of two characters ligature. The learner can write the word on the canvas using their device touch screen as shown in figure 5.10b. After writing the word, the learner can submit the attempt of the exercise. As feedback, the learner immediately receives a score out of 100 on the attempt as shown in figure 5.10c. If the score is less than 50, the learner can not move forward to another exercise and will have to re-attempt the exercise to improve.

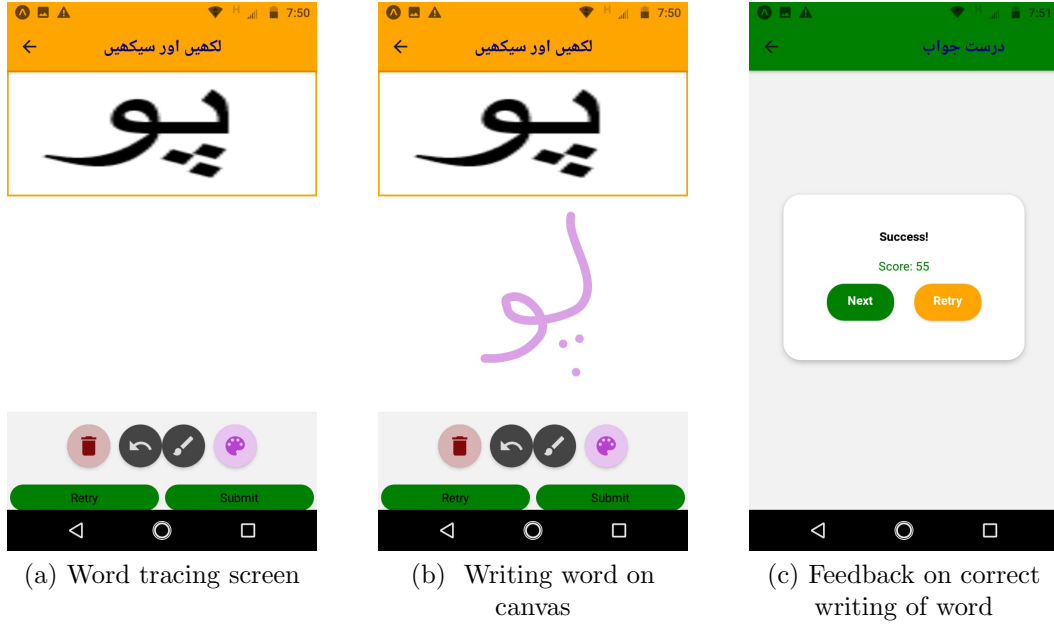


Figure 5.10: Urdu word tracing

5.2.3 Object Identification with Urdu Words

Figure 5.11 shows the screen if the learner selects to learn Object identification. Just like flashcards, the application shows the learner an image and a few Urdu words. The learner is to select the correct word which represents the image. The learner receives a full score and is directed to another exercise if the attempt is correct. If the learner selects the wrong word then the learner can not move forward to another exercise and will have to re-attempt the exercise to improve.



Figure 5.11: Object identification with Urdu Words

5.2.4 Urdu Words' Identification

Figure 5.12 shows the screen if the learner selects to learn Urdu words identification. Just like flashcards, the application shows the learner an Urdu word a few images. The learner is to select the correct image which represents the word. The learner receives a full score and is directed to another exercise if the attempt is correct. If the learner selects the wrong image then the learner can not move forward to another exercise and will have to re-attempt the exercise to improve.

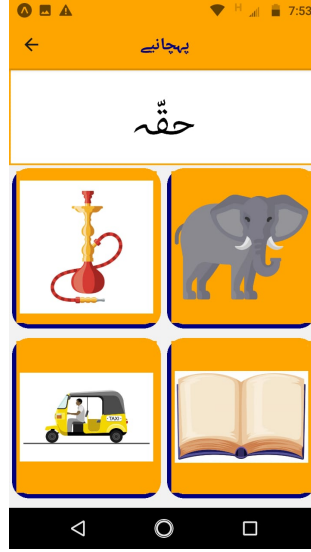


Figure 5.12: Urdu Word identification with Objects

5.3 Drawing Module

On selecting Drawing from modules, the learner is directed to the Drawing module screen as shown in figure 5.13a. Just like the Urdu module, the learner can see a sample drawing as a guide to draw. The learner can draw on the canvas using their device touch screen as shown in figure 5.13b. After drawing, the learner can submit the attempt of the exercise. As feedback, the learner immediately receives a score out of 100 on the attempt as shown in figure 5.13c. If the attempt of the learner is not correct and the score is less than 50, the learner can not move forward to another exercise and will have to re-attempt the exercise.

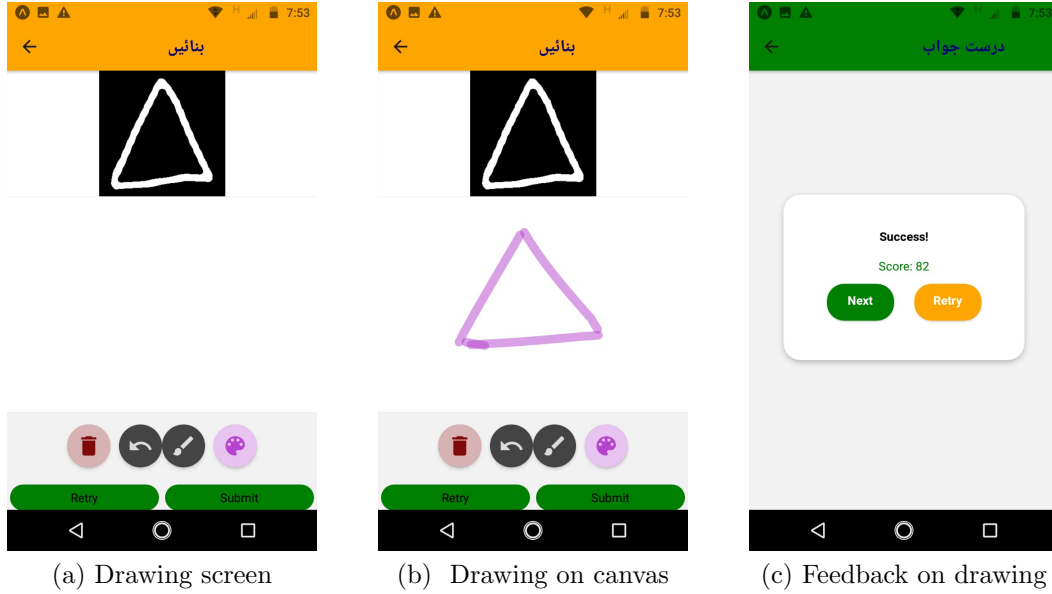


Figure 5.13: Drawing

5.4 Personalization of Exercises

For personalization, with every attempt of an exercise for Urdu, the app analyzes and keeps a record of the type of letters and words which the learner finds difficult to write, and based on the analysis the app provides similar practice exercises to the learner. Type of letters are clustered on basis of visual similarity and various features such as angles, and arcs in the letter. The application uses this record of exercises the learner is struggling with to then provide learner exercises of letters and words in the same cluster. App also provides similar personalization for drawing.

5.5 Dashboard

The app includes a performance dashboard to help parents, guardians, or educators to visualize the progress of the learner. The dashboard includes factors such as time spent by the learner during the last 7 days, average time spend per exercise, average feedback score during the last 7 days, and the number of exercises completed during the last 7 days. For easier visualization, the dashboard also shows time spent on

exercises of each module as a line graph, feedback score of each module as a line graph, and the number of exercises completed for each module as a bar graph.

6. Dataset

The dataset compiled by the team Aangan as a part of this project can be found on the github repository [here](#)

6.1 Data Collection for Urdu Characters

There is a lack of publicly available datasets for Urdu characters and Words. Therefore, the stroke data required for the process of Model training and Score generation was produced and compiled by the Team itself. For this purpose, an android application connected to the file system was set up. The application required participants to trace out the displayed character by hand or through a stylus with no time constraints in place. The application sign in required users to identify themselves and their age groups in order to store information of all users involved in the process. The emphasis during the data collection process was to collect a dataset from a diverse group of people in terms of their ability to write Urdu. Around 5000 samples were collected from different age groups ranging from 5 years to 20 years. Figure 6.5 shows the distribution of age groups involved in the data collection process.

So far, we have been able to collect more than 100 samples per character. Figure 6.1 displays the distribution of urdu characters that were collected.

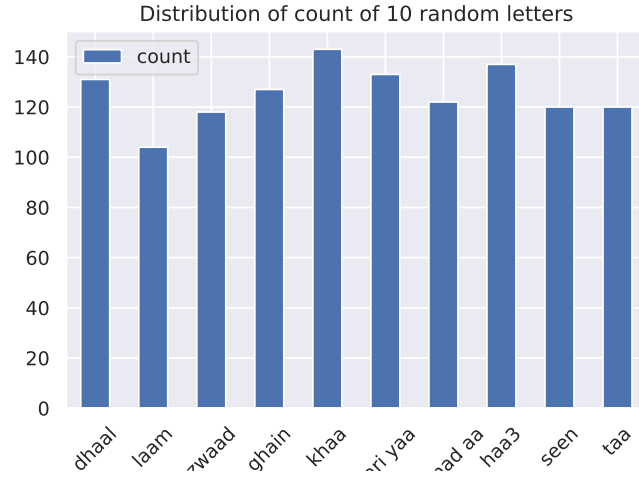


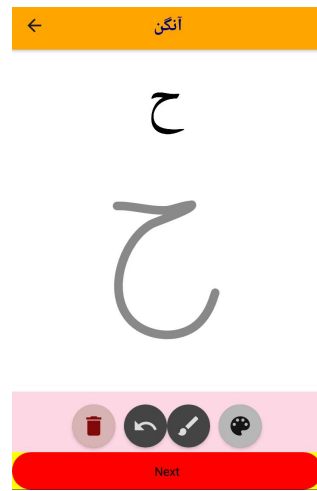
Figure 6.1: Distribution of Characters

The data is collected in the form of strokes and is stored in the git file system. The stroke sequence is arranged with respect to time, i.e. $x[t]$ is drawn right before $x[t + 1]$.

(a) Data Collection Application Homepage



(b) Character Display



(c) Traced Character

Figure 6.2: Data Collection Application

6.1.1 Data Scoring

The collected stroke samples were then converted into images and sent to native Urdu speakers to annotate scores in the range of 0 to 100 manually. Each sample was assigned to two different speakers for annotation to reduce the variance and increase uniformity. The final score for the sample was computed by averaging the score from both speakers. The scoring interface was built using Django and deployed using heroku.

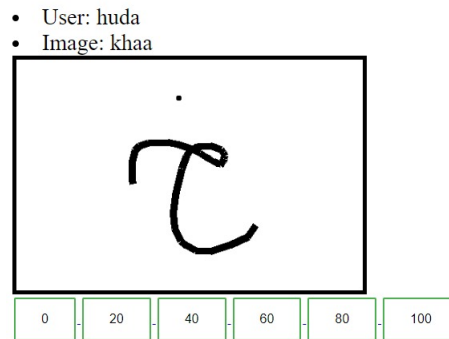


Figure 6.3: Interface for Scoring Application

Figure 6.4 displays the distribution of scores assigned by the native speakers to all collected samples. The distribution is left skewed suggesting a larger quantity of good or average samples in the dataset as compared to the bad ones.

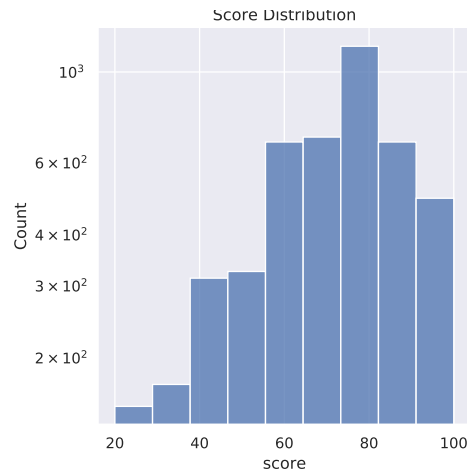


Figure 6.4: Distribution of Scores

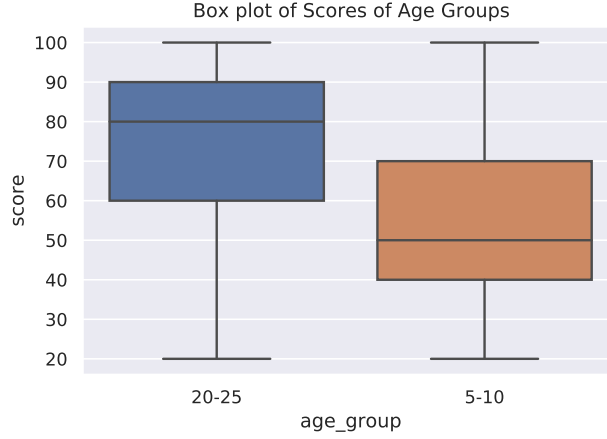


Figure 6.5: Distribution of Age Groups

Figure 6.5 shows the statistics for the score assigned by native Urdu speakers to different age groups. As expected, score assigned to samples from adults is higher as compared to samples collected by children.

6.1.2 Data Pre-processing

The data collected through this process was pre-processed as follows:

- All samples were scaled to maximum width and height of 128.
- All samples were aligned to the center of the canvas.
- The strokes were simplified using Ramer Douglas Peucker[24] algorithm with an epsilon value of 1.5.

All the existing datasets of Urdu characters are focused on handwriting recognition. Thus the dataset introduced by us is unique because it is centered on an application of e-learning Urdu handwriting. Furthermore, it is the first dataset that focuses on the diversity of the age-group of the participants.

6.2 Data Generation for Urdu Words

1. Urdu words are context sensitive i.e. their shape changes with respect to their position (start, middle, end) in a word. Therefore, during the process of data

collection we collect each character in each of its forms with respect to its position. Each character is carefully created such that it can be connected easily in the merging algorithm.

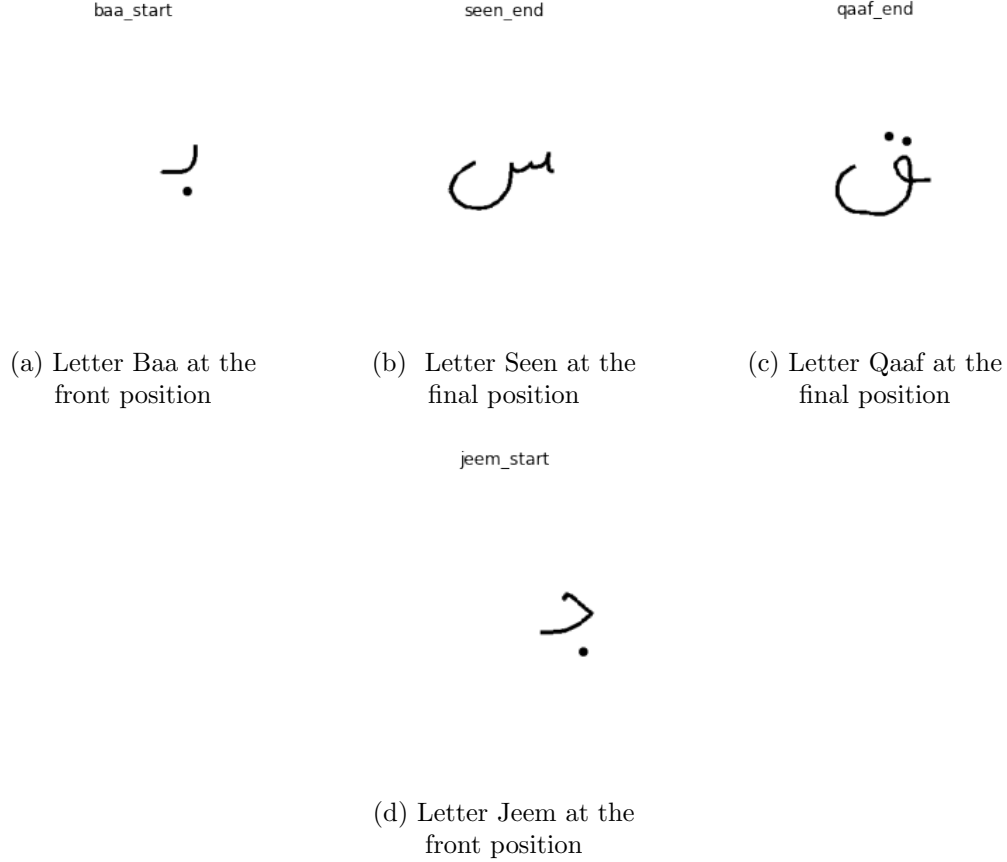


Figure 6.6: Word Data Samples

2. Each individual character is then scaled to have a uniform width and height of 300 and 380 respectively. All samples are further simplified using Ramer Douglas Peucker[24] algorithm. The dots (nuqte) are ignored in this step.
3. The individual characters are merged to create words. The algorithm combines the right most point of the starting character with the left most point of the next characters and then applies translations to merge them. The algorithm annotates meeting point for each word.

4. All the created words are then reprocessed to have uniform width and height. They are also at the same time mean centered.

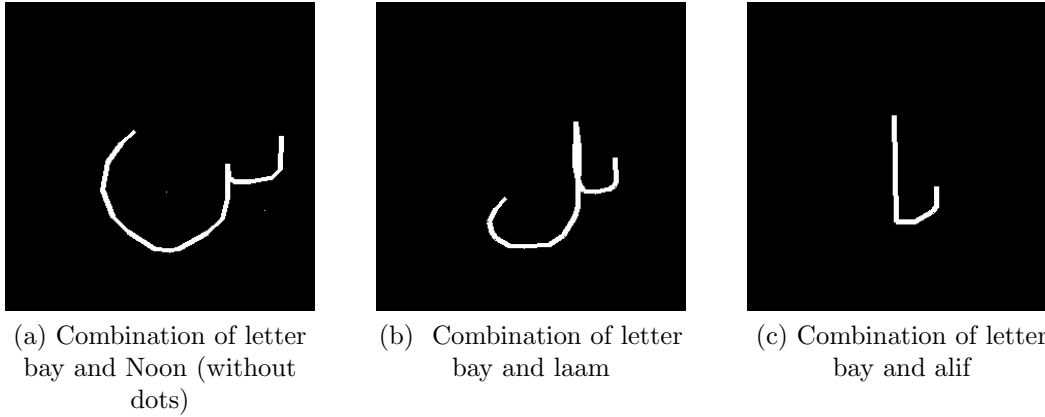


Figure 6.7: Word Data Samples

5. All the created words are then exported in the form of stroke and image.

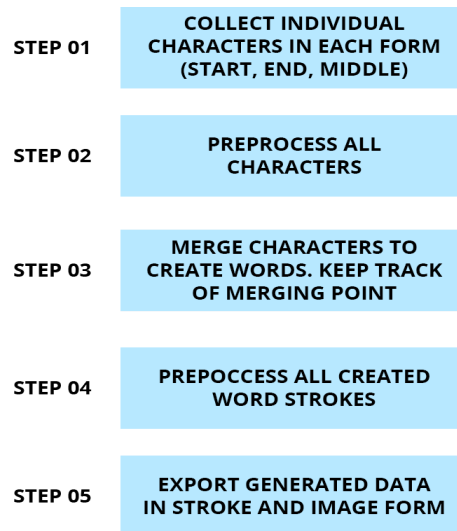


Figure 6.8: Data Generation Process

7. Experiments and Results

7.1 Methodology

Urdu Module Scoring

We propose two classes of models for scoring for Urdu. Following [31], one class of models is used to estimate an interclass score, while the second class of models is used for computing intra-class scores. The final computation of scores for each sub-module (Urdu character and Urdu words) uses one or more of these models.

7.1.1 Interclass Score

This class of models comprises two different CNN models trained on the same UCOM dataset [17]. The output labels associated with the samples were assigned according to the features assigned by [7]. These models work on image data, and hence the stroke from the application is first converted to an image and then sent to the model for estimation. The purpose of this model is to identify if the given sample contains the features associated with it. [7] divided the feature set into primary and secondary in which features associated with the main stroke of the letter are primary features, whereas dots or 'nuqte' constitute the secondary features for the respective letter. There are 32 primary features and 10 secondary features in total. One of the proposed models predicts primary features while the other predicts secondary features.

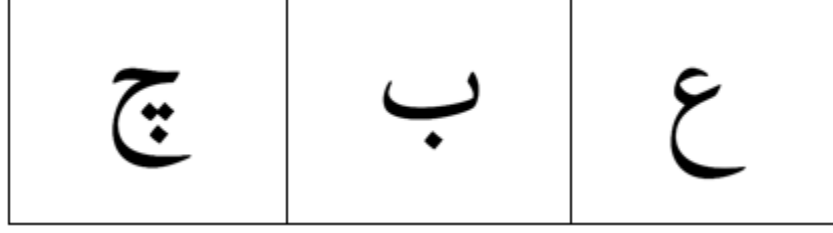


Figure 7.1: From Left to right: Urdu characters Chey (Primary features: Horizontal Line left to right, Semi Circle Up to down. Secondary Features: 3 dots, middle), Bey (Primary features: Start Vertical Line, Horizontal Line right to left, End Vertical Line. Secondary Features: 1 dots, below), and Ayn (Primary features: Start Semi Circle, Semi Circle Up to down. Secondary Features: None)

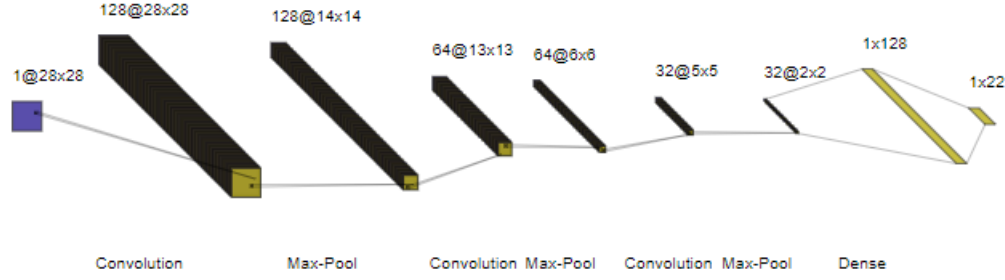


Figure 7.2: Primary Features Model Architecture

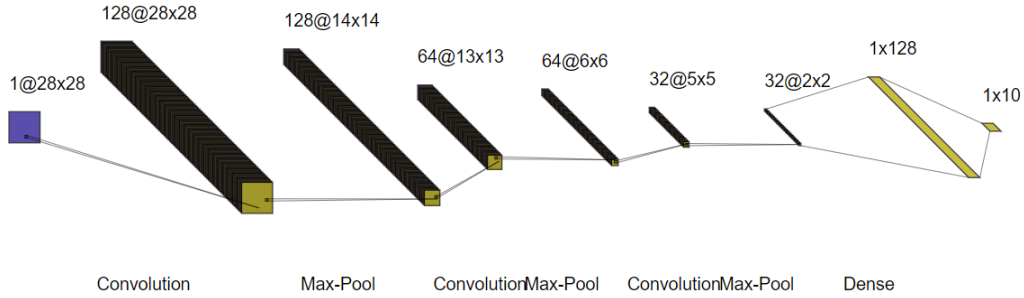


Figure 7.3: Secondary Features Model Architecture

Each model predicts a vector of probabilities associated with it for each feature set. The primary score is computed by averaging the predicted probabilities of

the features associated with that character. Thus for character ‘baa’ (7.1), we will average the probability of correct features (Start Vertical Line, Horizontal Line right to left, End Vertical Line). Similarly, a secondary score is computed, and the final score is given as:

$$Score = (primaryscore * 0.7) + (secondaryscore * 0.3)$$

We give more weightage to primary features, as according to our intuition and knowledge, primary features are harder to draw and hold more importance when it comes to recognizing the character as compared to secondary features.

7.1.2 Intra-class Score

This class of models is used to compute the intra-class score to see how the hand drawn sample compares to other samples in our data for the same character. For this purpose, a feature vector is computed for each sample in the dataset. The vector contains HBF features [11], a feature set specifically created for symbol recognition on electronic devices. The features introduced by [31] for scoring purposes are also utilized, which results in a feature vector of length 99 (49 HBF features + 48 Orientation features) for each sample in the dataset. The feature vectors are then used to estimate the intra-class score. For this estimation, we have tried two different approaches, one of which requires minimal data while the other requires a slightly larger amount of data.

Statistical Model

To train this model only the samples that have been assigned a score of 90 or higher are used. We pick 40 samples per category from those who qualify the score constraint. Thus a matrix with 99 columns (features) and 40 rows (samples) is computed for each category. We follow the approach introduced by [31] by learning the mean vector and covariance matrix for each category. At inference, the sample is first converted into a feature vector and then its distance is calculated from the distribution of its category using the Mahalanobis Distance [27].

Mahalanobis Distance is the distance between the mean vector and the given vector under a distribution. If the features have no correlation, the covariance matrix would be an Identity matrix and in that case, Mahalanobis Distance would simply compute the Euclidean distance between the mean vector and the given vector. Thus for a sample, the distance d is computed which is then converted into a score between

0 and 1 using the following equation,

$$Score = 1 - Percentile(d, category)$$

Where Percentile (d, category) gives the percentile of d in the distribution of its category.

Regression Model

With the dataset that we have collected and the manually annotated scores, a set of regression models are created for each category of character. Feature vectors described in section 7.2.2 serve as the input for this model, but before being into the model they are first normalized to have mean 0 and standard deviation 1. The output label (manually annotated score) is normalized in the same way as well. We use a variety of regression models provided by the scikit-learn [2] machine learning framework in Python. The dataset is first split into a training set (80%) and test set (20%) and then a set of regression models are trained. We use the default hyper-parameters corresponding to each model as provided by scikit-learn. We report the results of the model in the results section below.

7.1.3 Scoring for Urdu Characters

At the beginning of the learning phase, the interclass score, which establishes the legibility of the model, is of paramount importance, but after it the intra-class score, which establishes the quality of the handwritten sample holds more importance. Since we have sufficient data available for Urdu characters, we used two models and combined their scores. The models used for this module were the interclass model described in section 7.1.1 and the regression model described in 7.1.2.

7.1.4 Scoring for Urdu Words

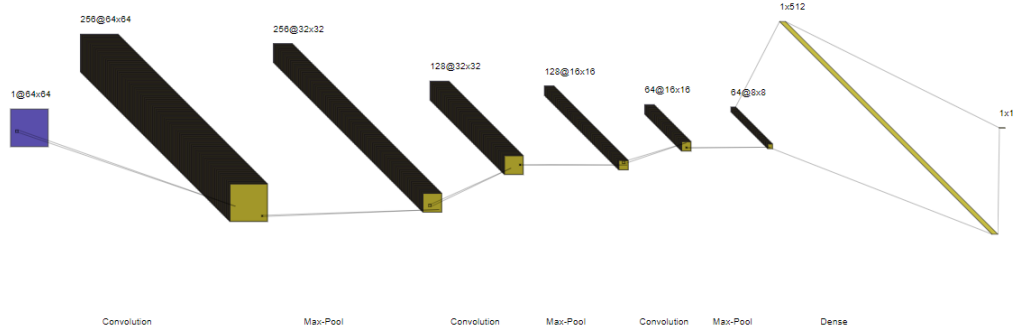


Figure 7.4: Urdu Words Segmentation Model Architecture

The data for Urdu words is not as abundant as the one for Urdu characters, which is why we were not able to train models for the interclass score. The final score for this module is computed using the model described in section 7.1.2 as it requires limited data. The sample Urdu word is first segmented into individual characters using an approach similar to [3], using the model shown in figure 7.4 and then the segmented characters are fed into the statistical model described in 7.1.2, which returns a score between 0 and 1. The figures below show this process for a sample two-letter word.

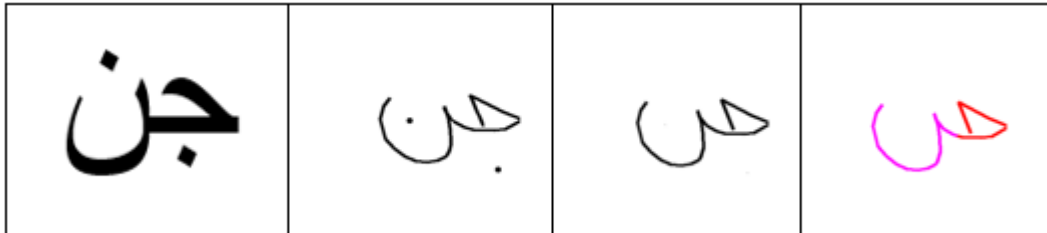


Figure 7.5: The figure shows the process of segmentation. From left to right: Reference image, sample stroke, sample stroke with dots removed, sample stroke segmented into two different strokes to be sent to the model described in Section 7.1.2

Drawing Module Scoring

The scoring for the Drawing module follows a logic similar to the method followed in section 7.1. We create two models, one of which computes the inter-class score and

the other computes the intra-class score. The first model is supposed to judge the legibility of the given sample while the second model is supposed judge the quality of the model. For construction of both the models the [13] data-set was used. For the drawing module we narrowed down to the following objects,

1. Circle
2. Triangle
3. Square
4. Cloud
5. Door
6. Eye
7. Flower
8. Apple
9. Star

7.1.5 Interclass Score

The interclass model was computed using a CNN model which was trained on [13] using the architecture shown in Figure 7.6. 20000 samples for each category mentioned above were used in training, with 70% in training set while the rest in test set. The QuickDraw! dataset comes in the form of stroke and images, but the stroke data was used. The stroke were converted to images using our stroke to image conversion algorithm which is used throughout the application. The purpose of converting the stroke data to image rather than using the dataset in the form of images was to reduce the data-shift as much as possible as the data in our application comes in the form of stroke, and it makes more sense to convert stroke to image using the same algorithm. Figure 7.7 illustrates the stepwise procedure for data set preparation.

STEP 01	STEP 02	STEP 03	STEP 04
RANDOMLY CHOOSE 20000 SAMPLES FOR EACH CATEGORY	PREPROCESS EACH STROKE	CONVERT EACH SAMPLE TO IMAGE	SPLIT DATA INTO TEST AND TRAINING SET
Each sample in the form of x,y coordinates. E.g. sample = [[1,2],[2,3],.....]	1) Zero scale all samples. 2) Scale all sample to constant size. 3) Simplify each stroke	Each sample is converted to its image representation. The dimensions of each image is (64, 64)	1) Training set = 70% 2) Test set = 30%

Figure 7.7: Stepwise procedure for the Dataset Preparation

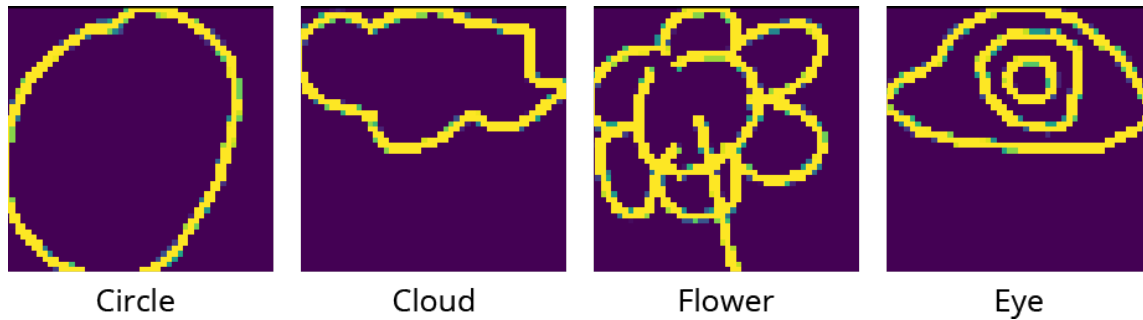


Figure 7.8: Few Samples from the training set

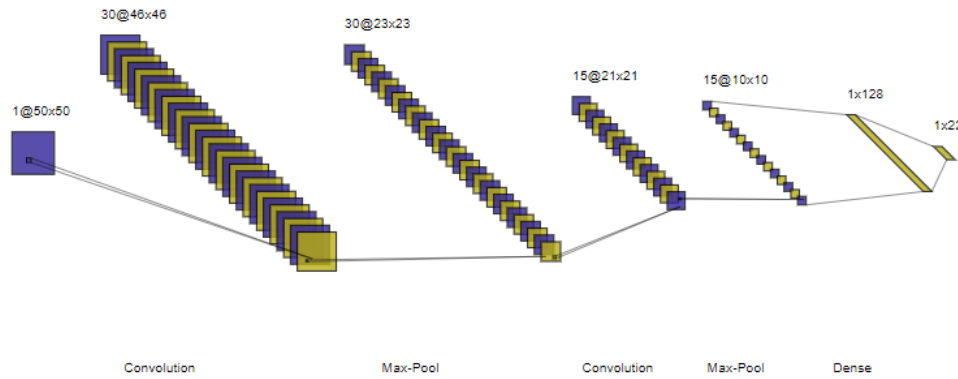


Figure 7.6: Architecture for the Drawing interclass Model

Figure 7.8 shows few samples from the training set after all the preprocessing steps have been applied. Figure 7.6 shows the architecture of the proposed model.

7.1.6 IntraClass Score

The interclass score is presented in a model quite similar to section 7.1.2. For this purpose, 45 perfect samples from each category were manually picked to ensure that the statistical model learns from the perfect sample. This was a challenge as the QuickDraw! dataset contains a lot of freehand samples and learning a distribution from those samples would be useless and hence the selection process is important. Thus, a co variance matrix and mean vector is computed for each category, which is then used to compute a score between 0 and 1 for each sample.

7.1.7 Score Combination

The combined score is simply the mean of interclass and intraclass score. The interclass model has extremely low variance while the intraclass model has higher variance, and thus the mean is the appropriate measure.

7.2 Results

The results for each of the 3 proposed models were validated on the dataset that we collected along with the manual annotation of the score. Since the purpose and nature of each model are different, we used different metrics to assess each model. It is important to note that the data on which the models were validated was not seen before by the model. The scores assigned by models were scaled to the range 0 - 100 before comparison, as the manual annotation used this range.

7.2.1 Inter-class Model

The purpose of this model is to identify if the sample is legible and recognizable. Hence for this purpose, the manually annotated was divided into two bins with score range $[0, 25)$ being ‘unrecognizable’ and the rest being ‘recognizable.’ The score predicted by the model is also converted into these bins, and thus the accuracy of this model is the proportion of images for which the correct bin was predicted. Table 7.1 shows the overall accuracy of the model on the overall dataset and on a random set of categories. As can be seen, the model is quite successful in predicting whether a sample is recognizable or not on completely unseen data.

Character	Accuracy (Test)
Jeem	85%
Twa	85.9%
Daal	90.9%
alif	53.8%
alif mad aa	58.8%
bari yaa	98.4%
cheey	94.2%
choti yaa	98.4%
dhaal	96.1%
Overall	84.3%

Table 7.1: The table shows the accuracy of the proposed model on a few random categories along with overall accuracy averaged over all categories



Figure 7.9: From left to right: Reference Image shown on the application for the character ‘Jeem’, Sample stroke classified as ‘unrecognizable’ by the interclass model, Sample stroke classes as ‘recognizable’ by the interclass model.

7.2.2 Intra-class Model

The purpose of this class of models is to estimate the quality of the given sample. These models were trained on 80% of the dataset and tested on 20% of the dataset. To validate the performance of the model described in 7.1.2, we use the following metrics: 1) Mean Absolute Difference: Mean absolute Difference between the model’s score and the manually assigned score. 2) Accuracy: Divide the score range 0-100 into bins of [‘Unrecognizable’, ‘Bad’, ‘Good’, ‘Excellent’]. [0, 25) score range is Unrecognizable, [25, 50) is ‘Bad’ and so on. Accuracy thus is the percentage of samples for which the model predicts the correct bin. 3) Range Accuracy: The

percentage of samples for which the predicted and actual scores have an absolute difference less than a range (e.g., 10 or 20).

Character	M.A.D (Test)	Accuracy (Test)	RA-10(Test)	RA-20(Test))
Qaaf	10.81	45.9%	70.2%	89.1%
Twa	11.22	55.5%	71.1%	86.6%
Swaad	11.28	57.6%	70.5%	91.0%
Jeem	11.64	58.2%	66.1%	91.0%
noon	11.914894	62.7%	69.1%	87.2%
faa	12.03	59%	76%	87%
bari-yay	12.66	62.2%	64.4%	77.7%
ayn	13.012048	46.9%	61.4%	87.9%
laam	13.12	75%	78.1%	90.6%
Overall	13.30	55.9%	65.4%	84.8%

Table 7.2: M.A.D: Mean Absolute Difference, RA: Range Accuracy - Summary of the results of the statistical model proposed in 6.2.1.

Table 7.2 shows the performance of this model, which is extremely promising especially considering the amount of data it requires. The absolute difference of 13.30 in score could also be justified by subjectivity as even in the annotated scores, the average difference between the score assigned by two different graders for the same sample was greater than 10.

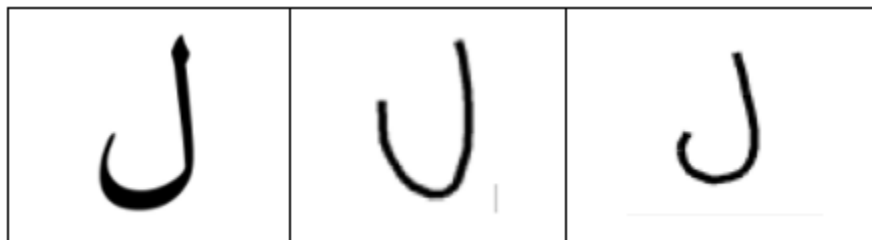


Figure 7.10: From left to right: Reference Image shown on the application for character ‘Laam,’ Sample stroke which was given a score of 53 while annotated score was 60, Sample stroke which was given a score of 97 while annotated score was 90.

Table 7.3 shows the performance of a variety of regression models proposed in 7.1.2. The results are averaged across all the categories. The metrics used are the same as those defined at the start of Section 7.2.2. The R-squared (R^2) values

Model	R^2	M.A.D(Test)	Accuracy(Test)	RA-10(Test)	RA-20(Test)
AdaBoost	80.2	12.71	60.1%	47.4%	79.4%
GradientBoosting	97	12.63	59.3%	46.6%	80%
RandomForest	91.6	12.51	58%	48.7%	79.1%
Decision Tree	99.9	14.82	56.9%	57.8%	83.1%
Ridge	77.1	17.75	52.9%	38.9%	66.8%
KNNeighbors	56.3	15.87	52.5%	48.7%	73.2%

Table 7.3: M.A.D: Mean Absolute Difference, RA: Range Accuracy - summarizes the results of the regression models proposed in 7.1.2

are an indication that there is a significant correlation between our independent variables (extracted features described in Section 7.1.2) and dependent variables (score). Gradient boosting is the pick of the models as it has the best test accuracy and the least error (mean absolute difference). The figure below shows the results of some samples passed through the GradientBoosting trained model.

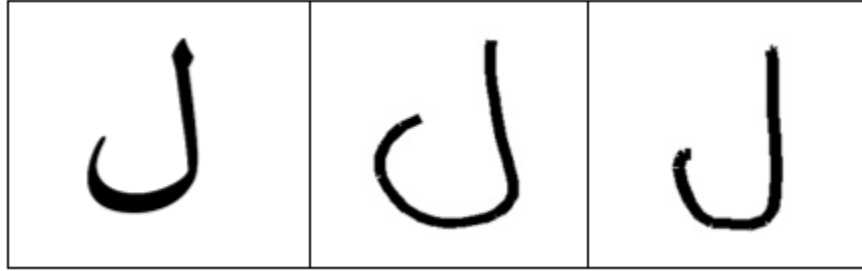


Figure 7.11: From left to right: Reference Image shown on the application for character 'Laam,' Sample stroke which our trained model gave a score of 72 while annotated score was 90, Sample stroke which was given a score of 67 while annotated score was 70.

8. Conclusion and Future Work

8.1 Conclusion

The project spanned two main fronts: product (application) development and research, where research was carried on both handwriting analysis of Urdu and analysis of hand-drawn sketches.

For Urdu, we explored the scope of stroke analysis and proposed recognition and scoring models for online Urdu handwritten characters and words. Thus far the research on handwriting analysis of both Urdu characters and words shows tremendous potential which can be used in elearning applications with far-reaching impact. The project also created and proposes a novel dataset for stroke-based and hand-written Urdu characters. Both the dataset and models provide promising results for the problem at hand and have set a pathway for further research in the domain. For drawing, we have introduced stroke analysis and explored the concept of automatic scoring of stroke-based sketches

As an application, the project holds significance as it introduces first-of-its-kind of application in Urdu and can set a pathway to introduce more e-learning applications in Pakistan that are focused on providing feedback and personalized experience to the learner.

8.2 Future work

Urdu is a very vast language and we do not want to limit learners to learn to write only one and two characters ligature. Research should be extended towards the recognition of three and four-character ligatures and words. Future work in Urdu also involves extending the dataset of Urdu characters to 10000 samples, extending

the dataset of Urdu words, and annotating the dataset for Urdu words. In this research, we extracted features using HBF49 and kaligo features. Research has to be carried out to find and explore other features such as beta elliptical features for better recognition results. For models on Urdu handwriting analysis, we currently have different models for recognition and scoring of different characters. Research has to be carried out to develop a CNN for generalized classification and score regression for Urdu Characters. For drawing, we can improve scoring methods for drawing for better results. In general, research can also be carried to explore models such as RNNs that take into account the sequence of a stroke.

We can also extend the work with regard to our application. The application is to be further developed to provide a more accurate sequential method to learn Urdu, as followed in Urdu qai'dah. Starting off, learners can be taught to write Urdu letters through rigorous trace-the-dots exercises, and then write without dots, on lines. Then, learners can be taught to write directly on plain canvas. To provide a greater learning experience, the application can also include engaging and lively video tutorials for children. The prototype application can be further tested by a large number of young learners to validate the current application and incorporate changes. Lastly, the application can be developed into a complete product including multiple subjects and can be launched into the Pakistani market as a unique e-learning application.

Appendix A. Code

Our repositories for the application can be found at this GitHub organization link. Repositories here are:

- File-system: To store and access exercise on Urdu and drawing and components of exercises such as sounds, sample images and corresponding objects.
- Backe-end of application made using Django: Backend repository gets deployed to heroku. All code that makes the backend of the application word can be found here.
- Mini application for dataset collection: The mini app is used to collect data. All code related to the app can be found here.
- Urdu dataset: Complete dataset collected on Urdu characters and words can be found here
- Scoring application: Each sample of data was scored by 2 individuals through this scoring app. All code related to the scoring app can be found here.
- Stroke analysis on Quick Draw dataset: All code used explore stroke analysis models on Google Quick Draw dataset can be found here.

References

- [1] Sherif Abdou et al. “Artificial Tutor For Arabic Handwriting Training”. In: Apr. 22, 2009.
- [2] *About us — scikit-learn 0.24.2 documentation*. URL: <https://scikit-learn.org/stable/about.html> (visited on 05/10/2021).
- [3] Asghar Ali. “Cursive-Text: A Comprehensive Dataset for End-to-End Urdu Text Recognition in Natural Scene Images”. en. In: 1 (Mar. 2020). Publisher: Mendeley. DOI: 10.17632/k5fz57zd9z.1. URL: <https://data.mendeley.com/datasets/k5fz57zd9z/1> (visited on 04/25/2021).
- [4] Hazrat Ali et al. “Pioneer dataset and automatic recognition of Urdu handwritten characters using a deep autoencoder and convolutional neural network”. en. In: *SN Applied Sciences* 2.2 (Jan. 2020), p. 152. ISSN: 2523-3971. DOI: 10.1007/s42452-019-1914-1. URL: <https://doi.org/10.1007/s42452-019-1914-1> (visited on 04/25/2021).
- [5] Adel Alimi. “ADAB database”. en. In: (Jan. 2021). Publisher: IEEE type: dataset. URL: <https://ieee-dataport.org/open-access/adab-database> (visited on 04/25/2021).
- [6] Saeed Anjum. “Early childhood education in Pakistan: Issues and challenges”. In: (2013).
- [7] Fareeha Anwar. “Online Urdu Handwritten Text Recognition For Mobile Devices Using Intelligent Techniques”. en_US. Accepted: 2019-05-16T05:50:47Z. Thesis. International Islamic University, Islamabad, 2019. URL: <http://prp.hec.gov.pk/jspui/handle/123456789/9542> (visited on 04/25/2021).
- [8] H. Bazine, A.M. Alimi, and N. Sherkat. “Generation and analysis of handwriting script with the beta-elliptic model”. In: *Ninth International Workshop on Frontiers in Handwriting Recognition*. ISSN: 1550-5235. Oct. 2004, pp. 515–520. DOI: 10.1109/IWFHR.2004.45.

- [9] Miguel Candeias et al. “Using Android Tablets to develop handwriting skills: A case study”. en. In: *Heliyon* 5.12 (Dec. 2019), e02970. ISSN: 2405-8440. DOI: 10.1016/j.heliyon.2019.e02970. URL: <https://www.sciencedirect.com/science/article/pii/S2405844019366290> (visited on 04/25/2021).
- [10] Mendeley Data. *Cursive-Text: A Comprehensive Dataset for End-to-End Urdu Text Recognition in Natural Scene Images*. 2020. URL: <https://data.mendeley.com/datasets/k5fz57zd9z/1>.
- [11] Adrien Delaye and Eric Anquetil. “HBF49 feature set: A first unified baseline for online symbol recognition”. en. In: *Pattern Recognition* 46.1 (Jan. 2013), pp. 117–130. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2012.07.015. URL: <https://www.sciencedirect.com/science/article/pii/S0031320312003317> (visited on 04/25/2021).
- [12] *Enabling My Robot To Play Pictionary — Proceedings of the 24th ACM international conference on Multimedia*. (Visited on 11/28/2020).
- [13] *googlecreativelab/quickdraw-dataset*. original-date: 2017-05-09T18:28:32Z. May 2021. URL: <https://github.com/googlecreativelab/quickdraw-dataset> (visited on 05/15/2021).
- [14] Mohit Gupta and Pulkit Mehndiratta. “Analysis and Recognition of Hand-Drawn Images with Effective Data Handling”. In: *Big Data Analytics*. Ed. by Sanjay Madria et al. Vol. 11932. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2019, pp. 389–407. DOI: 10.1007/978-3-030-37188-3_22. URL: http://link.springer.com/10.1007/978-3-030-37188-3_22 (visited on 11/28/2020).
- [15] David Ha and Douglas Eck. “A Neural Representation of Sketch Drawings”. In: *arXiv:1704.03477 [cs, stat]* (May 19, 2017). arXiv: 1704.03477. URL: <http://arxiv.org/abs/1704.03477> (visited on 11/28/2020).
- [16] Mujtaba Husnain et al. “Recognition of Urdu Handwritten Characters Using Convolutional Neural Network”. en. In: *Applied Sciences* 9.13 (Jan. 2019). Number: 13 Publisher: Multidisciplinary Digital Publishing Institute, p. 2758. DOI: 10.3390/app9132758. URL: <https://www.mdpi.com/2076-3417/9/13/2758> (visited on 04/25/2021).
- [17] *IAJIT - UCOM Offline Dataset-An Urdu Handwritten Dataset Generation*. URL: https://iajit.org/index.php?option=com_content&task=view&id=1136 (visited on 04/25/2021).

- [18] *ImageNet classification with deep convolutional neural networks — Communications of the ACM*. URL: <https://dl.acm.org/doi/abs/10.1145/3065386> (visited on 11/28/2020).
- [19] Yuchao Jiang. “Sketch image recognition using deep features”. In: (2017). Accepted: 2018-06-01T02:57:09Z. DOI: 10.4225/55/5b10b6edb84c6. URL: <https://digital.library.adelaide.edu.au/dspace/handle/2440/112588> (visited on 11/28/2020).
- [20] Kaggle. *Arabic Handwritten Characters Dataset*. 2020. URL: <https://www.kaggle.com/mloey1/ahcd1>.
- [21] Kaggle.com. *HaT: Urdu handwritten text dataset*. 2020. URL: <https://www.kaggle.com/hazrat/uhat-urdu-handwritten-text-dataset>.
- [22] Mohammed Khedher and Gheith Abandah. “Arabic Character Recognition using Approximate Stroke Sequence”. In: (Jan. 2002).
- [23] “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. In: (Sept. 2, 2014). arXiv: 1406.1078. URL: <http://arxiv.org/abs/1406.1078> (visited on 11/28/2020).
- [24] Bo Liu et al. “A Vector Line Simplification Algorithm Based on the Douglas–Peucker Algorithm, Monotonic Chains and Dichotomy”. In: *ISPRS International Journal of Geo-Information* 9.4 (2020), p. 251. DOI: 10.3390/ijgi9040251.
- [25] Mohamed Loey, Ahmed Elsayy, and Hazem El-Bakry. “Arab Kids Tutor (AKT) System For Handwriting Stroke Errors Detection”. In: *INTERNATIONAL JOURNAL OF TECHNOLOGY ENHANCEMENTS AND EMERGING ENGINEERING RESEARCH* 4 (Jan. 2016), pp. 42–49.
- [26] Makki Maliki, Naseer Al-Jawad, and Sabah Jassim. “Sub-word based Arabic Handwriting Analysis for Writer Identification”. In: vol. 8755. May 2013, p. 87550M. DOI: 10.1117/12.2017694.
- [27] G. J. McLachlan. “Mahalanobis distance”. en. In: *Resonance* 4.6 (June 1999), pp. 20–26. ISSN: 0973-712X. DOI: 10.1007/BF02834632. URL: <https://doi.org/10.1007/BF02834632> (visited on 05/10/2021).
- [28] Quickdraw.withgoogle.com. *Quick, Draw! The Data*. 2020. URL: <https://quickdraw.withgoogle.com/data>.
- [29] Patsorn Sangkloy et al. “The Sketchy Database: Learning to Retrieve Badly Drawn Bunnies”. In: *ACM Transactions on Graphics (proceedings of SIGGRAPH)* (2016).

- [30] Omar Seddati, Stephane Dupont, and Said Mahmoudi. “Deepsketch: deep convolutional neural networks for sketch recognition and similarity search”. In: *2015 13th international workshop on content-based multimedia indexing (CBMI)*. IEEE. 2015, pp. 1–6.
- [31] Damien Simonnet, Eric Anquetil, and Manuel Bouillon. “Multi-criteria handwriting quality analysis with online fuzzy models”. en. In: *Pattern Recognition* 69 (Sept. 2017), pp. 310–324. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2017.04.003. URL: <https://www.sciencedirect.com/science/article/pii/S0031320317301474> (visited on 04/25/2021).
- [32] Research Group on Urdu Language. *Offline Synthetic Urdu Dataset - Research on Urdu Language (RULE)*. 2020. URL: <https://sites.google.com/site/researchonurdulanguage1/databases/offline-urdu-database/offline-synthetic-urdu-database>.
- [33] Peng Xu et al. “Deep Learning for Free-Hand Sketch: A Survey and A Toolbox”. In: *arXiv:2001.02600 [cs]* (Apr. 26, 2020). arXiv: 2001.02600. URL: <http://arxiv.org/abs/2001.02600> (visited on 11/28/2020).
- [34] Rabiaa Zitouni, Hala Bezine, and Najet Arous. “Online Handwritten Arabic Scripts Recognition Using Stroke-Based Class Labeling Scheme”. en. In: *International Journal of Computational Intelligence Systems* 14.1 (Oct. 2020). Publisher: Atlantis Press, pp. 187–198. ISSN: 1875-6883. DOI: 10.2991/ijcis.d.201024.001. URL: <https://www.atlantis-press.com/journals/ijcis/125945570> (visited on 04/25/2021).